

ASPIRE: 用于机器人的智能体式 /Skills 发现

Runyu Lu^{1 2 * †}, Yubo Wu^{1 3 *}, Ethan Kou^{1 4 *}

Letian Fu^{1 4}, Wenli Xiao^{1 5}, Ajay Mandlekar¹, Yinzheng Xu¹

Guanya Shi⁵, Ken Goldberg⁴, Ang Chen², Mosharaf Chowdhury²

Yuke Zhu^{1 †}, Linxi “Jim” Fan^{1 †}, Guanzhi Wang^{1 †}

¹ NVIDIA, ² UMich, ³ UIUC, ⁴ UC Berkeley, ⁵ CMU

* 共同一作, † 项目负责人

<https://research.nvidia.com/labs/gear/aspire/>

摘要:

传统机器人编程以困难著称: 它需要编排多模态感知、管理复杂的物理接触动力学, 并处理多样的环境配置和执行失败。我们提出 ASPIRE (Agentic Skill Programming through Iterative Robot Exploration, 即通过迭代式机器人探索进行智能体式技能编程), 这是一个面向机器人的持续学习系统, 能够在代码即策略范式下自主编写和改进机器人控制程序, 同时将经验累积为可复用的技能库。ASPIRE 能够自动发现可复用技能, 这些技能可跨多个任务、仿真与真实世界场景以及不同具身形态持续发挥作用。ASPIRE 并不依赖固定的人工工程化流水线, 而是在一个开放式学习循环中运行, 该循环由三个关键组件构成: (1) 一个闭环机器人执行引擎, 暴露细粒度多模态轨迹 (例如感知叠加图、抓取候选、运动轨迹和碰撞反馈), 使智能体能够自主诊断失败、合成修复方案并验证结果; (2) 一个持续扩展的技能库, 将经过验证的修复提炼为可复用、可迁移的机器人知识; (3) 一个进化搜索过程, 生成多样的任务序列和控制程序, 并对其进行系统化调试, 以探索超越单轨迹改进的空间。随着 ASPIRE 遇到更多任务, 其不断增长的技能库使适应速度越来越快。因此, 在存在扰动的操作任务 (LIBERO-Pro) 上, ASPIRE 相比既有方法最高提升 77%; 在 Robosuite 的双臂交接任务上提升 72%; 在长时程家庭任务 (BEHAVIOR-1K) 上最高提升 32%。累积得到的技能库还带来了强大的零样本泛化能力: 在代表性的未见长时程任务 (LIBERO-Pro Long) 上, ASPIRE 达到 31% 的成功率, 尽管既有方法严重依赖测试时推理和重试, 其成功率仍仅为 4%, ASPIRE 明显优于这些方法。最后, 在仿真中发现的技能为仿真到现实迁移提供了初步证据; 即使具身形态和机器人 API 不同, 这些技能也显著减少了真实机器人编程工作量。

1. 引言

软件工程智能体的最新进展表明, 语言模型能够自主检查执行轨迹、定位失败、修订实现, 并通过与执行环境的反复交互持续改进 [Anthropic, 2025, OpenAI, 2025, anomalyco, 2025, Wang et al., 2025, Yang et al., 2024]。在机器人领域, 这一范式启发了 代码即策略系统, 这类系统将感知模块、规划 API 和控制原语组合为可执行的机器人程序 [Liang et al., 2023, Singh et al., 2023, Ahn et al., 2022, Huang et al., 2023, Mu et al., 2024, Fu et al., 2026]。由于机器人行为被显式表示为程序, 原则上它们可以通过交互反馈被检查、编辑、调试和改进。

然而, 现有机器人编码智能体从根本上仍受限于朴素的执行环境, 因为这些环境只提供粗粒度的任务级反馈。调试机器人程序本身就具有挑战性, 因为失败可能来自许多相互作用的组件, 包括多模态感知、运动规划、抓取生成、接触动力学和长时程任务协调。一次失败的执行尝试可能表明任务没有成功, 但无法说明根因究竟是感知错误、抓取不稳定、规划错误, 还是下游恢复失败。如果没有细粒度诊断轨迹, 智能体很难判断应检查哪些证据、如何定位失败, 或应尝试哪种修复策略。

此外, 现有系统不会跨任务累积经验。一旦任务完成, 已发现的修复和恢复策略就会被丢弃, 而不是被整合为可复用技能。因此, 解决第一百个任务的智能体实际上并不比解决第一个任务时更有经验。

人类机器人工程师采用的是根本不同的方法。当机器人程序失败时, 他们会回放执行过程, 检查感知输出和运动轨迹, 定位失败的子系统, 修订实现, 并内化可复用的恢复策略。随着时间推移, 调试经验会累积为可迁移知识, 包括抓取恢复启发式方法、导航策略、提示配方, 以及可跨任务泛

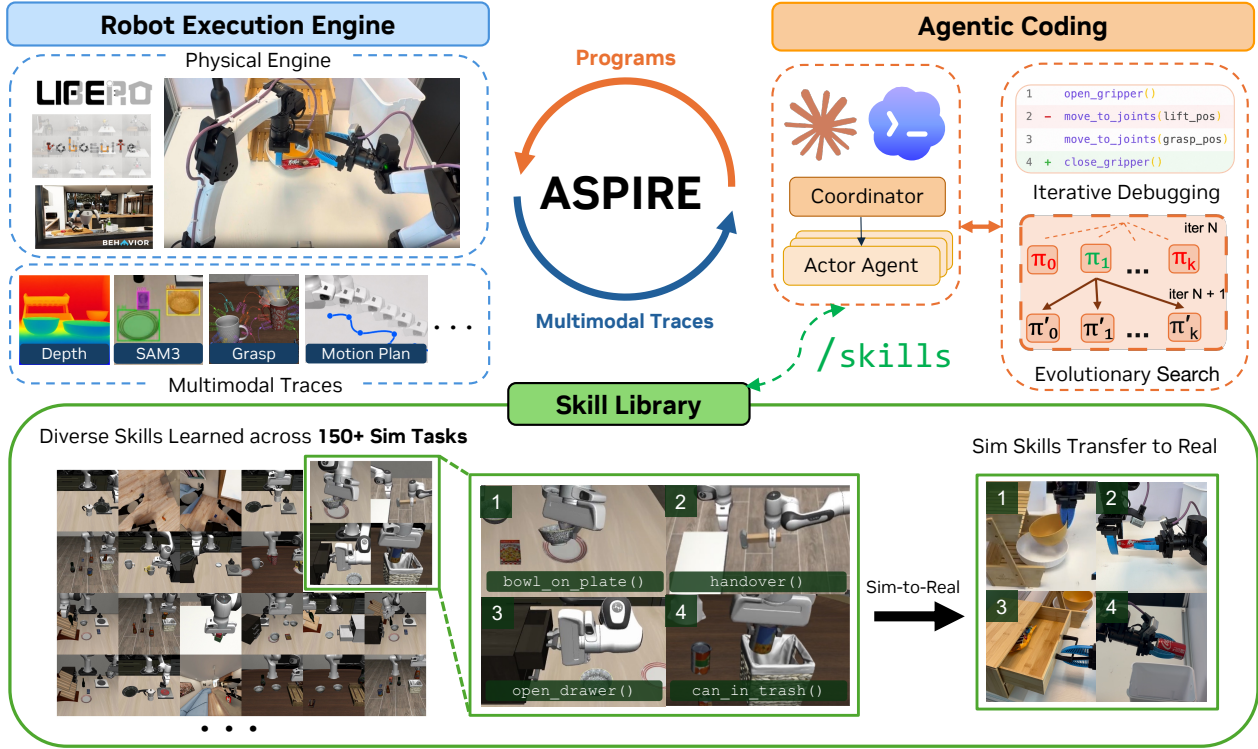


图 1: **Aspire 系统概览**。一个协调器为每个任务生成一个行动者智能体（编码智能体），从而支持跨任务并行学习。每个行动者通过与机器人执行引擎进行迭代式调试来改进和验证机器人程序；该引擎暴露每个原语的多模态轨迹，用于失败归因和修复。进化搜索对多样的候选程序 ($\pi_0, \dots, \pi_k$) 进行采样，将每个程序送入引擎，并基于幸存程序和残余失败轨迹来条件化下一代程序 ($\pi'_0, \dots, \pi'_k$)。协调器将经过验证的修复写入共享技能库，供未来行动者作为上下文内指导检索；在仿真中发现的技能也可以被调整为跨具身形态的真实机器人编程指导。

化的程序化修复。这种可复用知识的积累，是人类机器人程序员随着时间推移逐步变得更高效的关键原因。

在本文中，我们提出 ASPIRE，这是一个自我改进的持续学习机器人系统，能够在代码即策略范式下自主编写和改进机器人控制程序，同时将经验累积为可复用技能库。ASPIRE 并不运行在固定的感知-规划-执行流水线中，而是在一个开放式学习循环中运行；在该循环中，智能体自主决定如何检查执行轨迹、诊断失败、合成修复、验证修正后的行为，并将成功的恢复模式整合为可跨任务迁移的持久技能。

ASPIRE 的概览如图 1 所示。ASPIRE 由三个组件构成。首先，**闭环机器人执行引擎**用每个原语的执行轨迹取代粗粒度执行展开级反馈。对于每一次感知、规划、抓取和控制调用，执行引擎都会尽可能记录观测、输入、输出和视觉证据。这些丰富的多模态轨迹使智能体能够选择性地检查关键原语日志，逐步定位失败，并通过重新执行来验证修复。其次，ASPIRE 维护一个不断增长的 **技能库**，将经过验证的修复提炼为可复用、可迁移的机器人知识，可作为未来任务的上下文内指导被检索。第三，ASPIRE 采用一种 **进化搜索过程**，生成多样的任务序列和控制程序，通过迭代式调试和并行改进探索超越单轨迹自我改进的空间。这些组件共同构成了一个自我改进的机器人系统，其性能会随经验增长而扩展：ASPIRE 看到的任务越多，其技能库就越大，并且越能迁移到新任务、更长时程的行为，以及真实世界机器人场景；在这些场景中，相似的失败-恢复模式会在不同具身形态下出现。

在经验结果上，ASPIRE 在多样的短时程和长时程机器人基准上展现出很强的自我改进能力。相

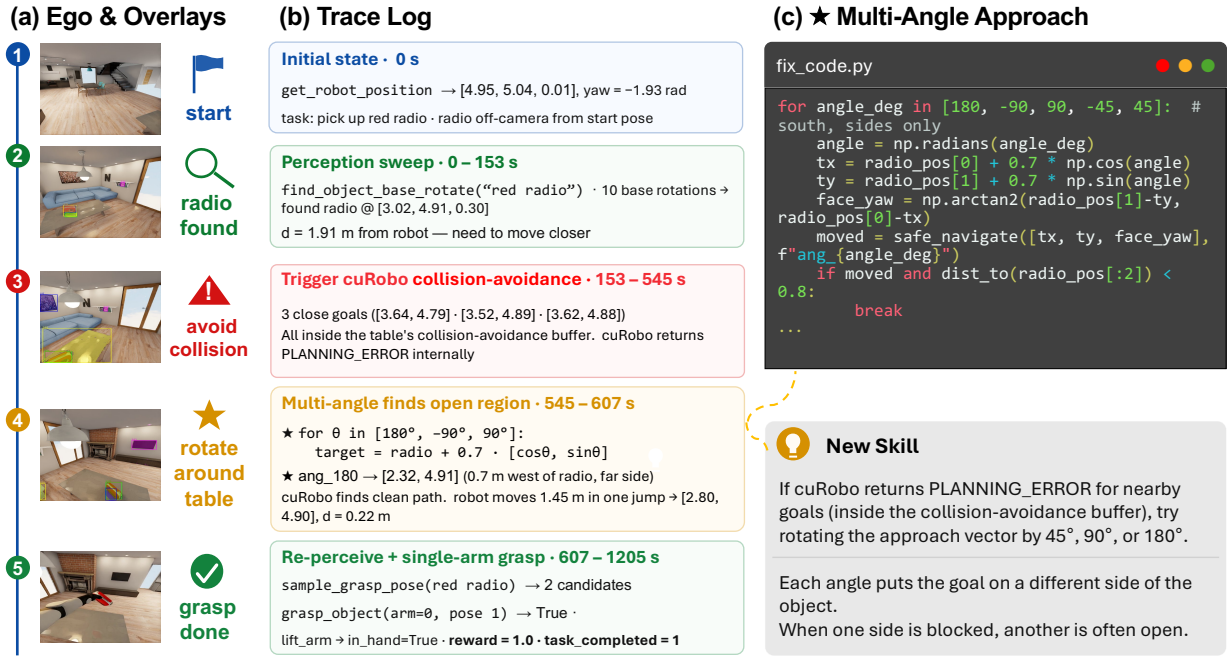


图 2: 机器人执行引擎。在 BEHAVIOR-1K NAVIGATE-AND-PICK-UP-RADIO 任务上的轨迹引导调试。(a) 自我视角关键帧和叠加图显示, 机器人定位到了收音机, 但未能接近它。(b) 原语轨迹将失败定位到反复出现的 PLANNING_ERROR: 候选导航目标落在桌子的避碰缓冲区内。(c) 智能体用多角度接近例程修补程序, 从可达一侧重新感知收音机, 并完成抓取。经过验证的修复被接纳为可复用的 Multi-Angle Approach 技能。

较于既有编码智能体 [Fu et al., 2026], ASPIRE 在 LIBERO-Pro [Zhou et al., 2025] 扰动套件上将成功率最高提升 77 个百分点, 在 Robosuite [Zhu et al., 2020] 接触丰富的操作任务上最高提升 72 个百分点, 并在具有程序化生成布局的 BEHAVIOR-1K [Li et al., 2024] 长时程家庭任务上最高提升 32 个百分点。基于在 LIBERO-90 [Liu et al., 2023] 上累积的技能库, ASPIRE 零样本迁移到 LIBERO-Pro Long [Zhou et al., 2025], 并达到 31% 的成功率; 相比之下, 既有方法尽管依赖测试时推理和重试, 成功率仍停滞在 4%。我们还在一台真实双臂机器人上评估了三个仿真中发现的技能, 结果表明, 将仿真中发现的技能作为上下文内指导进行检索, 可以减少真实世界推理词元, 并在没有技能迁移的朴素调试完全失败的场景中生成成功程序。

2. 方法

ASPIRE 由三个组件构成, 它们共同形成一个开放式学习循环: (1) 机器人执行引擎 (§2.1) 暴露按原语记录的多模态轨迹, 用于失败归因, 并执行智能体编写的修复以进行闭环验证; (2) 技能库 (§2.2) 将经过验证的修复积累为可复用技能; 以及 (3) 进化搜索过程 (§2.3) 将探索拓展到单条轨迹修复之外。随着 ASPIRE 遇到更多任务, 它的技能库会不断增长, 使未来任务能够继承累积的修复和可复用策略。

如图 1 所示, ASPIRE 采用协调器-行动者架构。中央协调器管理共享技能库, 并将行动者编码智能体分派到各个任务; 每个行动者则在机器人执行引擎中编写、执行、诊断并修复机器人程序。行动者不会交换完整聊天历史或原始执行轨迹。相反, 可迁移经验被提炼进技能库, 使每个行动者的上下文窗口能够聚焦于任务规格、当前程序, 以及与当前失败相关的结构化执行轨迹。

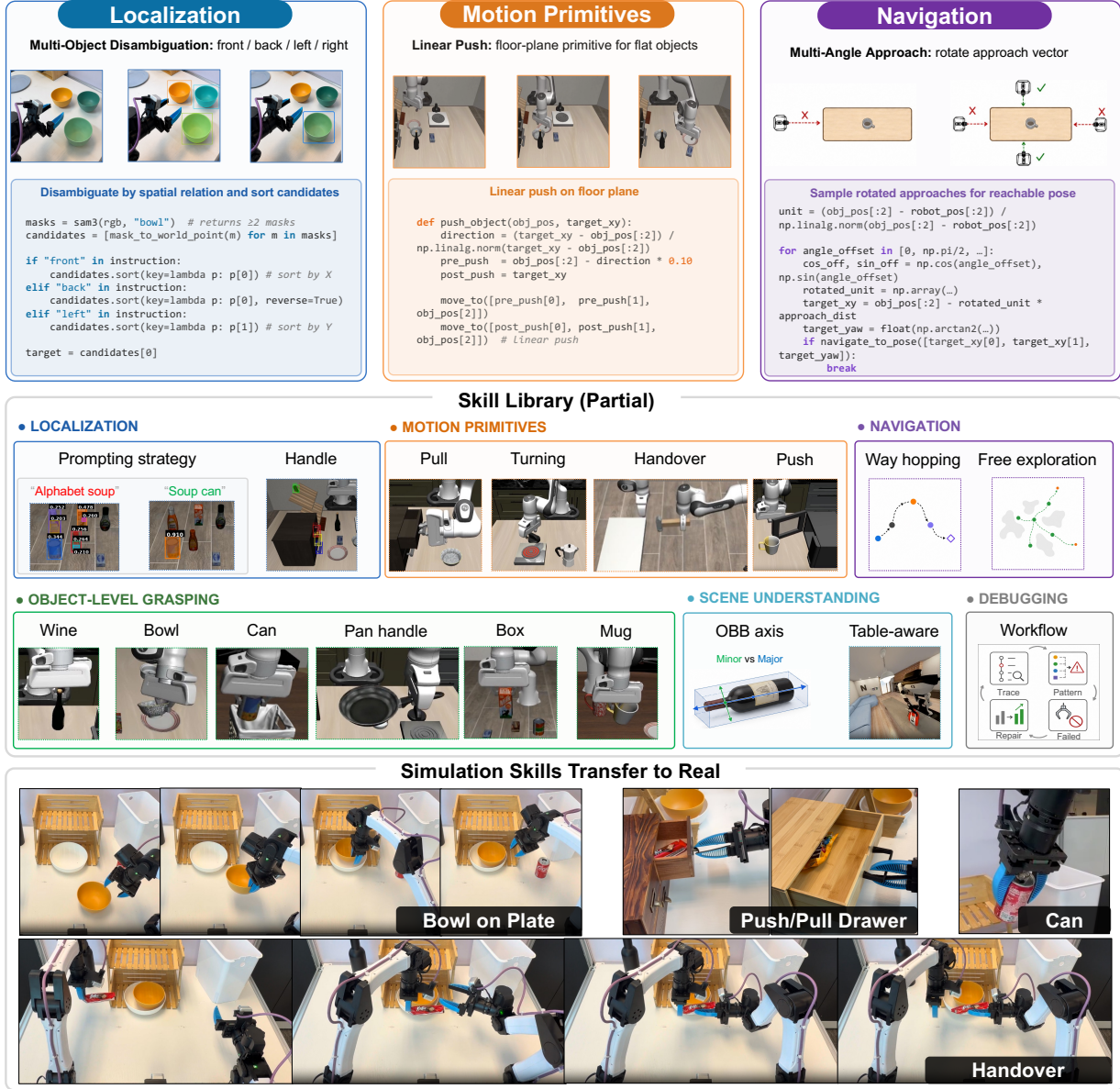


图 3: 技能库。ASPIRE 将经过验证的、由智能体发现的修复知识存储为可复用的上下文内技能，而不是一组固定的人类编写原语。顶部：代表性条目展示关于定位消歧、运动原语构造和导航恢复的已学习技能。中部：该库在异构类别上增长，包括定位、导航、运动原语、对象级抓取、场景理解和调试 workflow。底部：在仿真中发现的若干技能被用作真实机器人编程的上下文内指导，提供了技能能够跨具身迁移的证据。更多技能示例和提示见附录 A。

2.1. 机器人执行引擎

具身编码智能体需要执行证据来调试机器人程序。以往方法通过固定的、人类设计的接口暴露这些证据，通常表现为人工整理的场景级摘要或一组预定义观测。这带来了一种权衡：证据太少会掩盖出错原语，而过多的原始视觉上下文又会分散智能体对造成失败的因果链的注意力。

ASPIRE 将这种固定反馈通道转变为开放式调试环境。机器人执行引擎会记录感知、规划和控制调用的按原语多模态轨迹，将轨迹暴露给编码智能体，并执行智能体编写的修复以进行闭环验证。对每个原语调用，轨迹会存储被调用的 API、输入与输出、返回状态，以及相关多模态证据，例如 RGB 关键帧、叠加图、抓取候选、对象位姿和运动规划结果。智能体不会收到完整视频帧；引擎会保留每个原语调用前后的帧以及对应叠加图和返回值，使智能体能够聚焦于与失败相关调用周围的证据。

图 2 放大展示了一个 BEHAVIOR-1K NAVIGATE-AND-PICK-UP-RADIO 调试片段。自我视角关键帧显示，机器人找到了收音机，反复无法接近它，然后在改变接近方向后成功。原语轨迹将失败定位出来：感知成功并返回收音机位姿，但反复的 `navigate_to_pose` 调用返回 `PLANNING_ERROR`。通过检查导航返回值和相关日志，智能体发现生成的导航目标太靠近桌子边界，大约位于桌边 20 厘米以内。这会触发避碰并导致规划器失败。因此，失败并非因为检测或抓取收音机，而是因为桌子的碰撞约束下目标位姿不可行。

修复直接来自这一诊断。智能体没有更改感知提示或抓取原语，而是编写了一个多角度接近例程，在收音机周围采样替代导航目标，并执行能避开碰撞缓冲区的接近方向。执行引擎暴露证据，验证修补后的程序，并使智能体能够分析由此产生的日志、形成假设并做出有针对性的修复决策。

2.2. 技能库

程序失败会在任务之间反复出现，但可复用知识很少是完整的任务程序。ASPIRE 的技能库存储异构修复知识：定位启发式、感知提示、抓取约束、导航恢复策略、运动原语、场景理解例程和调试工作流。我们不预先规定这种分类。技能从经过验证的修复中归纳得到：编码智能体根据执行轨迹诊断失败、修补程序、在调试配置上验证修复，而协调器只将可复用模式接纳进共享库。

每项技能都以紧凑的上下文内指导形式存储，包括失败特征、适用条件、修复策略，以及在有用时附带一个代表性代码草图。图 3 展示代表性条目和由此形成的库广度。对于图 2 中的收音机任务，被接纳的技能是导航恢复模式，而不是完整的收音机拾取程序：当规划器错误在障碍物边界附近反复出现，原因是采样的目标位姿落入碰撞缓冲区时，先围绕对象采样替代接近方向，再重新尝试感知和抓取。这种表示使未来的行动者能够复用经过验证的修复，而不是通过测试时推理重新发现它们，支持向更难仿真任务的零样本迁移，并提供一种机制，使选定的仿真中发现的技能能够跨具身泛化并迁移到真实机器人。

行动者报告结构化发现，概括失败模式、经过验证的修复以及潜在可迁移的修复模式。协调器审计这些发现，验证其符合允许的 API 策略，并仅将通过调试验证的可复用修复提升到共享技能库。附录 E.1 提供用于行动者报告和协调器引导技能接纳的提示。

2.3. 进化搜索

仅靠轨迹引导调试可能会坍缩为局部修复循环，其中智能体反复修补同一种失败策略，而不是探索从根本上不同的任务解决方式。ASPIRE 使用进化搜索来拓宽可执行机器人程序的探索，鼓励多样化的修复假设和任务策略。

在每一轮中，基于技能库，编码智能体会提出由 K 个候选程序组成的种群；这些候选以表现最好的先前程序和先前评估中的失败轨迹为条件。每个候选都在机器人执行引擎中执行，产生任务结果以及新的诊断轨迹。下一轮随后以表现最好的程序及其剩余失败模式为条件，使搜索能够探索不同策略，而不是反复细化同一解决方案。

算法 1 对程序进行进化搜索

输入: 任务 τ , 程序 P^0 , 集合 $S_{\text{dbg}}, S_{\text{val}}$, 技能库 $\mathcal{L}$, 智能体 M , 预算 (K, T) , 阈值 θ

记号: $\text{EXECUTE}(P, S) = (r, Z)$ 返回得分 r 和轨迹包 Z 。

```

1:  $(r^*, Z^0) \leftarrow \text{EXECUTE}(P^0, S_{\text{dbg}})$ ;  $P^* \leftarrow P^0$ 
2:  $\mathcal{H} \leftarrow \{(P^0, r^*, Z^0)\}$ 
3: for  $i = 1, \dots, T$  do
4:    $\{P_i^k\}_{k=1}^K \leftarrow$ 
     PROPOSEREPAIRS( $M, \tau, \text{Top3}(\mathcal{H}), \mathcal{L}, \mathcal{H}$ )
5:   for  $k = 1, \dots, K$  do
6:      $(r_i^k, Z_i^k) \leftarrow \text{EXECUTE}(P_i^k, S_{\text{dbg}})$ 
7:   end for
8:    $\mathcal{H} \leftarrow \mathcal{H} \cup \{(P_i^k, r_i^k, Z_i^k)\}_{k=1}^K$ ;  $k^* \leftarrow \arg \max_k r_i^k$ 
9:   if  $r_i^{k^*} > r^*$  then
10:     $(P^*, r^*) \leftarrow (P_i^{k^*}, r_i^{k^*})$ 
11:   end if
12:   if  $r^* \geq \theta$  then
13:     break
14:   end if
15: end for
16:  $(r_{\text{val}}, Z_{\text{val}}) \leftarrow \text{EXECUTE}(P^*, S_{\text{val}})$ 
17:  $\mathcal{G} \leftarrow$ 
  EXTRACTVALIDATEDPATTERNS( $\mathcal{H}, P^*, r_{\text{val}}, Z_{\text{val}}$ )
18: 返回  $(P^*, r_{\text{val}}, \mathcal{G})$ 

```

搜索目标是机器人程序本身。候选通过闭环执行来选择；搜索结束后，如果经过验证的修复能够跨环境变化和任务泛化，则被接纳进技能库。当某个候选解决调试配置，或搜索预算耗尽时，搜索终止。算法 1 总结了进化搜索过程。流水线详情见附录 E.4。

3. 实验

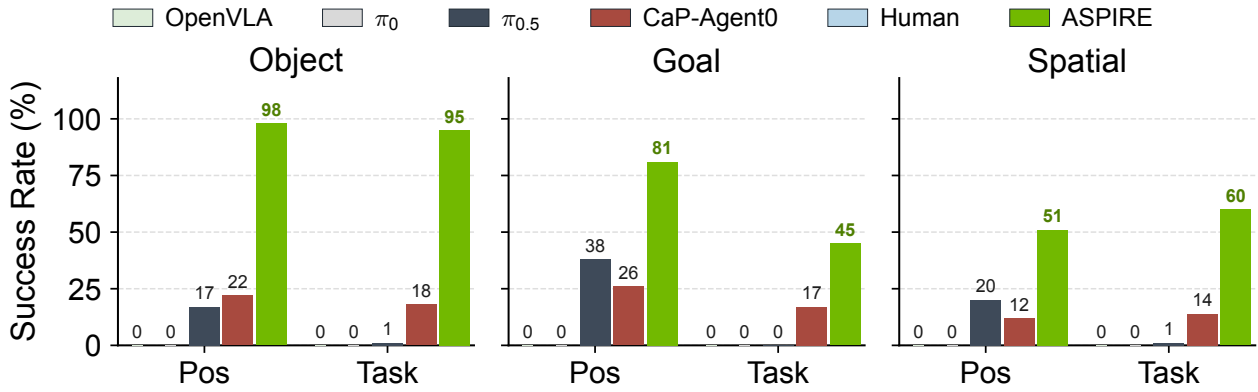
3.1. 实验设置

对于所有仿真基准，我们使用 Claude Code [Anthropic, 2025] 搭配 Claude Opus 4.6 和 100 万词元上下文窗口作为编码智能体。该智能体在 CaP-X [Fu et al., 2026] 中编写可执行的 Python 机器人程序；CaP-X 是一个构建在 MuJoCo Playground [Zakka et al., 2025] 之上的开源代码即策略 (code-as-policy) 框架，提供用于感知、几何和运动规划的机器人编程 API。所有实验中的智能体、环境和 API 集都保持固定。

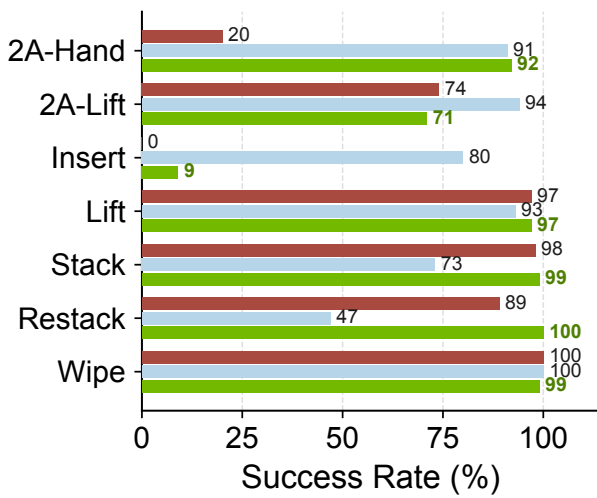
对于真实机器人技能迁移研究 (§3.6)，我们在双臂 YAM 操作平台上使用 reasoning-xhigh 模式下的 OpenAI Codex GPT-5.5。我们选择 ASPIRE 在基于 Franka 的仿真中编译出的三项技能：苏打罐拾取、碗放置到盘子上，以及抽屉推/拉。我们将这些技能作为上下文内指导提供给真实机器人编码智能体。真实机器人设置使用与仿真不同的具身形态和 API，但会暴露 ASPIRE 的多模态执行轨迹，使智能体能够在调试循环中无需特定任务的人类指导，自主运行、检查并调试程序。我们比较带有和不带有相应仿真发现技能的运行，度量首次成功所需的词元数以及留出真实机器人成功率。

3.2. 基准与基线

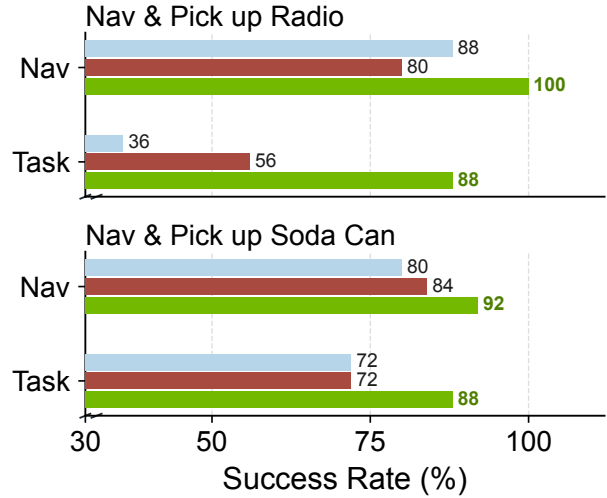
我们在三个基准族上评估 ASPIRE: LIBERO-Pro [Zhou et al., 2025] 用于评估对象、目标和空间扰动下的短时程鲁棒性；Robosuite [Zhu et al., 2020] 用于评估接触密集型单臂和双臂操作；BEHAVIOR-1K [Li et al., 2024] 用于评估 NAVIGATE-AND-PICK-UP-SODA-CAN 和 NAVIGATE-AND-PICK-UP-RADIO 上的长时程家庭移动操作。我们的主要编码智能体基线是 CaP-Agent0 [Fu et al., 2026]，它使用视觉差分、预定义技能库和逐回合测试时重试。我们还与端到端 VLA 策略进行比较，包括 OpenVLA [Kim et al., 2024]、 π_0 [Black et al., 2025] 和 $\pi_{0.5}$ [Physical Intelligence et al., 2025]。对于零样本迁移，我们使用在 LIBERO-90 [Liu et al., 2023] 上累积的技能库评估留出的 LIBERO-Pro Long 任务。



(a) **LIBERO-Pro** [Zhou et al., 2025]: 在每个套件/扰动上, 对 10 个任务 $\times$ 50 个留出环境种子的成功率做宏平均。对于每个任务, ASPIRE 在种子 51-65 上学习并收集技能, 并在种子 1-50 上评估一个生成程序。



(b) **Robosuite** [Zhu et al., 2020]: 每个任务在 100 次留出试验上的成功率。ASPIRE 在种子 101-125 上学习, 并在种子 1-100 上为每个任务评估一个生成程序; 2A 任务为双臂任务。



(c) **BEHAVIOR-1K** [Li et al., 2024]: 两个家庭任务上的长时程移动操作。ASPIRE 在种子 26-35 上学习, 并通过增量块执行在种子 1-25 上评估; Navigation 和 Task 成功率分别报告。

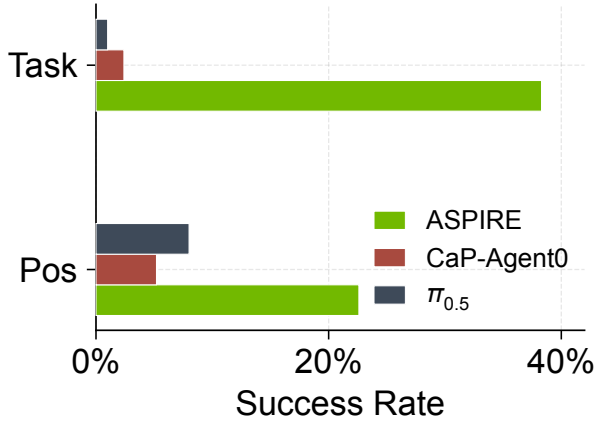
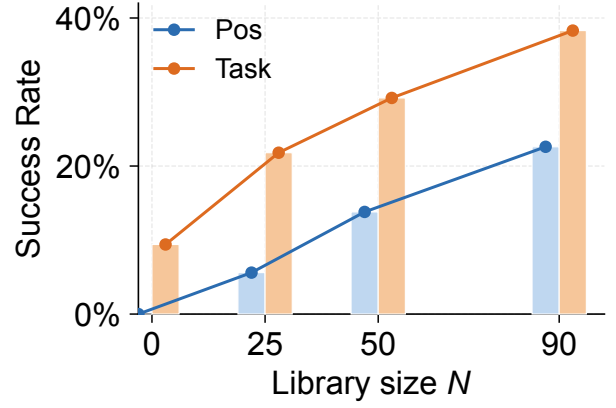
图 4: **Aspire** 在三个基准族上优于既有编码智能体和端到端 VLA。 (a) LIBERO-Pro 上的短时程操作; (b) Robosuite 上的接触密集型操作; (c) BEHAVIOR-1K 上的长时程移动操作。ASPIRE 在留出种子上为每个任务评估一个生成程序, 而 CaP-Agent0 则通过测试时推理和重试, 为每个种子重新生成一个单独程序。ASPIRE 提升了所有基准上的性能, 其中若干结果超过了人类专家编写的程序。详细的逐任务结果见附录 B 和 D.1。

3.3. 评估协议

在所有基准中, 一个环境种子固定一个任务实例, 包括物体位姿、干扰物以及机器人/物体初始状态。我们使用不相交的调试种子和评估种子: ASPIRE 在一个小型调试划分上学习, 然后在规模更大的留出评估种子上报告成功率, 其中每个 LIBERO-Pro/Robosuite 任务使用一个生成程序; 而 CaP-Agent0 则通过测试时推理和重试, 为每个种子重新生成一个单独程序。对于 BEHAVIOR-1K 评估, ASPIRE 使用增量块执行, 从当前多模态轨迹生成下一个代码块。

3.4. 主要评估结果

图 4 总结了主要评估。“Human”表示程序由人类专家编写。在 LIBERO-Pro 上, ASPIRE 在三个套件上都优于所有基线: 对 Pos 和 Task 扰动轴取平均后, ASPIRE 相比每个套件中最强基线, 在 Object

(a) 完整 $N=90$ 技能库的零样本结果与基线对比。

(b) 随着技能库规模增加，零样本成功率提升。

图 5: LIBERO-Pro Long 上的跨任务零样本迁移。在 LIBERO-90 上累积的技能提升了留出长时程任务上的零样本性能。图 (a) 将完整 $N=90$ 技能库与基线进行比较。图 (b) 展示了技能库规模增加时的 Pos/Task 成功率。所有成功率均是在每个轴的 10 个任务上做宏平均。逐任务结果见附录 C。

上提升 77%，在 Goal 上提升 41.5%，在 Spatial 上提升 42.5%。 $\pi_{0.5}$ 在若干位置扰动上强于 OpenVLA 和 π_0 ，但仍远低于 ASPIRE，并且在任务改述下基本崩溃。在 Robosuite 上，ASPIRE 在较容易的接触密集型任务上保持接近饱和的性能，并将双臂交接从 20% 显著提升到 92%。在 BEHAVIOR-1K 上，ASPIRE 在导航和任务成功率上都优于人类和 CaP-Agent0，其中相对 CaP-Agent0 最大的任务层面提升出现在 NAVIGATE-AND-PICK-UP-RADIO 上，从 56% 提升到 88%。

3.5. 向未见任务的零样本迁移

我们评估在 LIBERO-90 上累积的修复技能能否零样本迁移到留出的 LIBERO-Pro Long 任务。ASPIRE 从 $N \in \{0, 25, 50, 90\}$ 个源任务构建技能库快照，其中 $N=0$ 是空库设置， $N=90$ 是完整套件。对于每个留出任务，ASPIRE 生成一个程序，并在多个种子上评估它，不进行额外调试、重试或特定任务的技能库更新。

图 5(a) 在两个 LIBERO-Pro Long 迁移轴上比较了完整 $N=90$ 技能库与既有基线：ASPIRE 在位置扰动上达到 23% 成功率，在任务扰动上达到 38%，并在两个轴上都优于 CaP-Agent0 和 $\pi_{0.5}$ 。图 5(b) 表明，随着技能库规模增长，成功率稳定提升，这说明来自短时程任务的已验证修复为更长时程的组合提供了可复用的机器人知识。

3.6. 跨具身形态的真实机器人技能迁移

我们评估在仿真中学到的技能能否减少不同具身形态的真实机器人上的调试工作量。这种迁移并不是直接部署策略：真实机器人使用自身的感知、标定和控制栈，编码智能体仍必须通过真实世界执行反馈来适配程序。问题在于，检索仿真发现的技能是否能提供有用的上下文内指导，从而减少达到成功程序所需的真实世界调试量。

表 1 表明，迁移技能会稳定降低调试成本，而它们对最终成功率的影响取决于任务。碗放置在两种设置中都成功，但使用技能检索时消耗的词元更少；苏打罐举起从 13/20 提升到 19/20，同时总词元数降低了近一个数量级；抽屉操作在技能指导下达到 11/20 成功率，而无技能基线耗尽了更大的词元预算，却未能产生成功的评估程序。这些结果表明，经选择的、由失败导出的技能可以跨具身形态和 API 变化指导真实机器人程序合成，而不只是记住特定于仿真的代码。

表 1: **真实机器人跨具身形态技能迁移**。对于每个任务，我们比较从 ASPIRE 的技能库中检索相应仿真发现技能和不检索该技能时的真实机器人调试。词元数统计到首次得到成功的真实机器人程序为止；对于未达到成功的运行，则统计到调试预算耗尽为止。成功率报告生成程序的评估试验结果。

任务	输出词元 (M) ↓		总词元 (M) ↓		成功率 ↑	
	无技能	有技能	无技能	有技能	无技能	有技能
将碗放到盘子上	0.05	0.04	8.65	5.11	20/20	20/20
举起苏打罐	0.18	0.03	61.94	6.58	13/20	19/20
打开/推抽屉	1.33	0.36	334.917	81.67	0/20	11/20

3.7. 消融研究

机器人执行引擎与进化搜索。我们在 LIBERO-Pro 上消融 ASPIRE 的两个关键组件：机器人执行引擎，它暴露密集执行轨迹并验证任务层面的修复；以及进化搜索，它在机器人执行引擎调试之后探索额外的修复候选。图 6(a) 和图 6(b) 将成功率分解为不含任一组件的基础系统、加入机器人执行引擎带来的增益，以及进化搜索带来的额外增益。机器人执行引擎提供最大的平均提升，将宏平均成功率从 14% 提升到 62%；进化搜索进一步改善剩余困难任务，在两个组件同时使用时达到 72%。

进化搜索迭代。图 6(c) 跟踪了随着进化搜索预算增加，低表现任务上的性能变化。成功率在前几轮稳定提升，这表明采样多个修复假设可以快速找回单轮调试遗漏的替代方案。随后曲线继续更缓慢地上升，说明额外轮次对剩余困难案例仍有帮助，但收益递减。

4. 相关工作

智能体式机器人控制。机器人控制已经通过端到端视觉-语言-动作策略，以及将感知、规划和控制 API 组合起来的可执行程序得到研究 [Ahn et al., 2022, Brohan et al., 2023, Octo Model Team et al., 2024, Kim et al., 2024, Black et al., 2025, Mu et al., 2024, Physical Intelligence et al., 2025, NVIDIA et al., 2025, Sun et al., 2026, Li et al., 2026, Fu et al., 2026]。软件工程智能体为代码提供了相关的编写-执行-调试循环 [Jimenez et al., 2024, Wang et al., 2025, Yang et al., 2024, Wang et al., 2024a, Chen et al., 2024, Anthropic, 2025, OpenAI, 2025, anomalyco, 2025]。ASPIRE 建立在可执行程序范式之上，但聚焦于持久的具身改进：智能体可以自由选择原语级多模态轨迹，编写修复，进行验证，并跨任务保存成功经验。

自我改进智能体与技能库。已有工作通过开放式记忆、技能库和自演化技能仓库改进 LLM 智能体 [Wang et al., 2023, Tzifas and Kasaei, 2024, Peng et al., 2024, Fu et al., 2025, Zhang et al., 2026, Xie et al., 2026]。其他系统使用 LLM 生成奖励、课程、环境或搜索候选 [Ma et al., 2024a, Yu et al., 2023, Xie et al., 2024, Liang et al., 2024, Ma et al., 2024b, Du et al., 2023, Wang et al., 2024b, Romera-Paredes et al., 2024, Novikov et al., 2025, Guo et al., 2025, 2026, Cao et al., 2026]。不同于仅保存成功记忆、文本反思或用于下游策略训练的奖励函数，ASPIRE 存储的是从已归因的具身失败中提取的、经过验证的修复知识。其技能在内容和准入上都是开放式的：它们来自智能体自身的调试经验，覆盖异质类别 (§2.2)，并作为未来程序修复的上下文内指导被复用。进化搜索 (§2.3) 还会在可复用模式被纳入技能库之前，进一步拓宽可执行程序修复的空间。

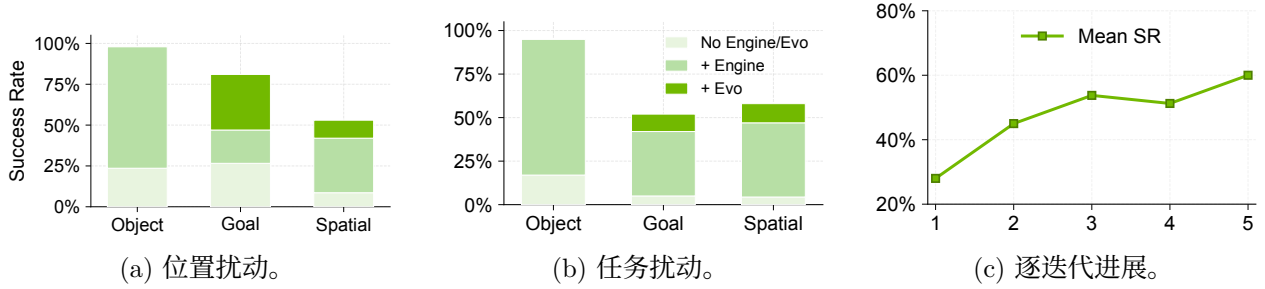


图 6: LIBERO-Pro 上机器人执行引擎与进化搜索的消融。图 (a) 和图 (b) 用堆叠柱展示位置扰动和任务扰动：不含机器人执行引擎或进化搜索的基础系统、加入机器人执行引擎带来的收益，以及进化搜索带来的额外收益。平均而言，机器人执行引擎提供的收益最大，将宏平均成功率从 14% 提高到 62%；进化搜索进一步改进了剩余的困难任务。图 (c) 展示低表现任务在进化搜索迭代中的平均进展。逐任务结果见附录 D。

5. 局限性

ASPIRE 有若干重要局限。第一，尽管若干在仿真中发现的技能支持在不同具身形态上进行自主真实机器人调试，我们的系统尚不是一个完全自主的真实世界终身学习者。不同于仿真中廉价且可程序化的成功检查和场景重置，真实世界部署仍然需要稳健的成功检测、安全重置、安全监控和标定维护。未来工作应闭合这一评估与重置循环，从而在更广泛的真实世界任务套件中扩展仿真到现实技能迁移。第二，我们的方法依赖一个冻结的前沿 LLM（Claude Opus 4.6，具有 100 万词元上下文窗口）来解释多模态轨迹、编写程序修复，并提出进化搜索候选；我们尚未验证更小或更弱的 LLM 是否能够维持同样的调试循环。第三，ASPIRE 使用预定义的感知、规划和控制原语 API 来编写程序。这个 API 使调试变得可处理且安全，但也限制了智能体能够表达的行为：如果任务需要暴露原语之外的感知、控制或交互能力，智能体要么必须以低效方式近似这些能力，要么依赖人类扩展 API。未来系统应研究智能体如何安全地提出、验证并纳入新的机器人原语。第四，技能库目前优先保存经过验证的可复用修复，但尚未完全解决长期记忆管理。随着技能库增长，一些条目可能会对新任务变得陈旧、过于具体、冗余或具有误导性，这可以解释零样本迁移中的非单调趋势。为了在规模化时保持技能库有用，需要更稳健的检索、剪枝、排序和重新验证机制。最后，调试与进化搜索循环计算开销很大，每个任务都会消耗大量 LLM 调用以及仿真器或机器人执行展开；要扩展到非常大的任务套件，需要更廉价的 LLM 推理、更具样本效率的搜索，或更强的既有修复复用机制。

6. 结论

我们提出 ASPIRE，这是一个持续学习机器人系统，能够自主编写和改进机器人控制程序，同时将经验累积为可复用技能库。ASPIRE 在一个由三个组件组成的开放式学习循环中运行：一个暴露细粒度多模态轨迹的闭环机器人执行引擎、一个持续扩展并将经过验证的修复提炼为可迁移知识的技能库，以及一个探索多样任务序列和控制程序的进化搜索过程。在多样基准上，ASPIRE 显著优于现有 VLA 和编码智能体基线，对未见长时程任务展现出很强的零样本迁移能力，并提供了初步证据，表明即使机器人具身形态和 API 不同，仿真中发现的技能也可以跨具身形态迁移，从而显著降低真实机器人编程的词元成本。

致谢

我们感谢 Nadun Ranawaka、Jimmy Wu、Matin Furutan、Haotian Lin、Abhi Maddukuri、Yulu Gan、Matin Nikoui 和 Yuqi Xie 在开源支持、真实机器人基础设施、论文建议与指导、网站发布以及实验设备方面提供的帮助。我们也感谢 NVIDIA GEAR、UMich SymbioticLab、UsesysLab、UC Berkeley AUTOLab 和 CMU LeCAR Lab 的成员给予的友善支持。

References

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Goper, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.
- anomalyco. Opencode: The open source ai coding agent, 2025. URL <https://github.com/anomalyco/opencode>. Accessed May 2026.
- Anthropic. Claude code: Best practices for agentic coding, 2025. URL <https://www.anthropic.com/engineering/claude-code-best-practices>. Accessed April 2026.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.
- Shiyi Cao, Ziming Mao, Joseph Gonzalez, and Ion Stoica. K-search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.
- Yuqing Du, Olivia Watkins, Zihan Wang, Cédric Colas, Trevor Darrell, Pieter Abbeel, Abhishek Gupta, and Jacob Andreas. Guiding pretraining in reinforcement learning with large language models. In *International Conference on Machine Learning (ICML)*, 2023.
- Honghao Fu, Junlong Ren, Qi Chai, Deheng Ye, Yujun Cai, and Hao Wang. Vistawise: Building cost-effective agent with cross-modal knowledge graph for minecraft. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- Max Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026.
- Ping Guo, Chenyu Zhu, Siyuan Chen, Fei Liu, Xi Lin, Zhichao Lu, and Qingfu Zhang. Evoengineer: Mastering automated CUDA kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025.
- Ping Guo, Chao Li, Yinglan Feng, and Chaoning Zhang. Code evolution for control: Synthesizing policies via LLM-driven evolutionary search. *arXiv preprint arXiv:2601.06845*, 2026.
- Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.
- Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, et al. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning (CoRL)*, 2024.
- Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. Eurekaverse: Environment curriculum generation via large language models. In *Conference on Robot Learning (CoRL)*, 2024.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Yecheng Jason Ma, William Liang, Hung-Ju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. Dreureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024b.
- Yao Mu, Juntao Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, Peize Sun, Haibao Yu, Chao Yang, Wenqi Shao, Wenhao Wang, Jifeng Dai, Yu Qiao, Mingyu Ding, and Ping Luo. Robocodex: Multimodal code generation for robotic behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2024.
- Alexander Novikov, Ngô Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz,

- Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonetop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.
- OpenAI. Codex: Ai coding partner from openai, 2025. URL <https://openai.com/codex/>. Accessed May 2026.
- Shaohui Peng, Xing Hu, Qi Yi, Rui Zhang, Jiaming Guo, Di Huang, Zikang Tian, Ruizhi Chen, Zidong Du, Qi Guo, Yunji Chen, and Ling Li. Hypothesis, verification, and induction: grounding large language models with self-driven skill learning. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI’24/IAAI’24/EAAI’24. AAAI Press, 2024. ISBN 978-1-57735-887-9. doi: 10.1609/aaai.v38i13.29376. URL <https://doi.org/10.1609/aaai.v38i13.29376>.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL)*, 2025.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- Xiaoquan Sun, Zetian Xu, Chen Cao, Zonghe Liu, Yihan Sun, Jingrui Pang, Ruijian Zhang, Zhen Yang, Kang Pang, Dingxin He, Mingqi Yuan, and Jiayu Chen. Atomvla: Scalable post-training for robotic manipulation via predictive latent world models. *arXiv preprint arXiv:2603.08519*, 2026.
- Georgios Tzifas and Hamidreza Kasaei. Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024a.

- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brenman, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Zackory Erickson, David Held, and Chuang Gan. Robogen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024b.
- Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-skill: Building self-evolving skill repository for generalizable robotic manipulation. *arXiv preprint arXiv:2603.02623*, 2026.
- Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023.
- Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A. Kahrs, Carmelo Sferrazza, Yuval Tassa, and Pieter Abbeel. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.
- Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. Skillflow: benchmarking lifelong skill discovery and evolution for autonomous agents, 2026. URL <https://arxiv.org/abs/2604.17308>.
- Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020.

A. 技能库详情

本附录将图 3 中展示的代表性条目扩展为技能库分类体系。每个条目将列出：(i) 从触发失败轨迹中提取的问题；(ii) 编码情境检索条件的适用时机守卫；(iii) 已验证的修复片段；以及 (iv) 产生该条目的来源任务。

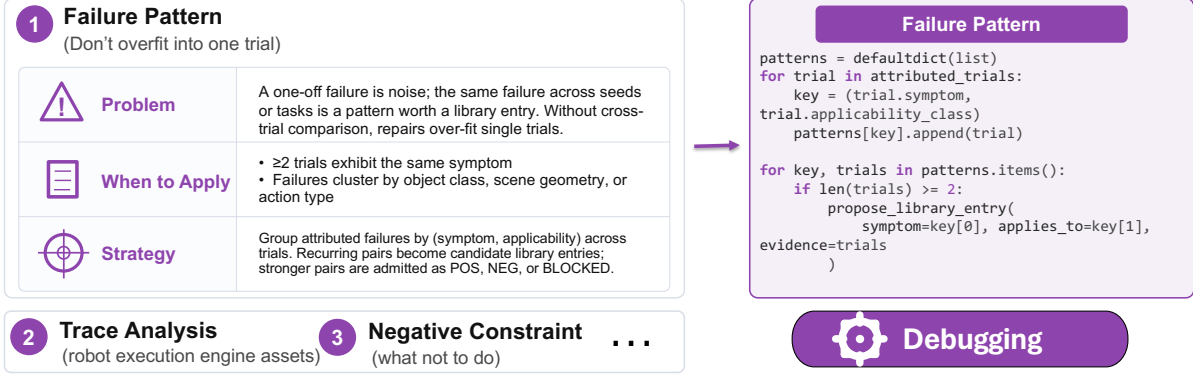


图 7: 调试技能。编码可复用调试策略的代表性技能库条目，包括失败特征、适用时机守卫以及已验证的修复草图。

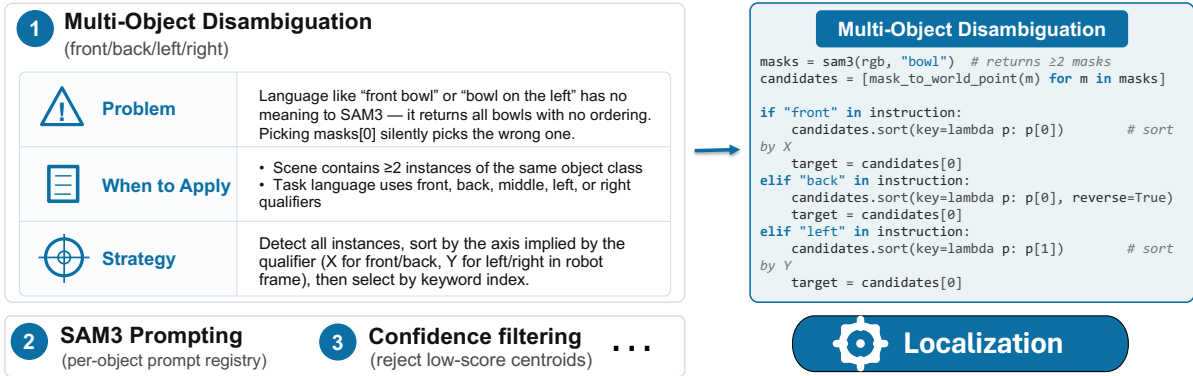


图 8: 定位技能。将含糊语言和对象指称落地为鲁棒感知与定位例程的代表性条目。

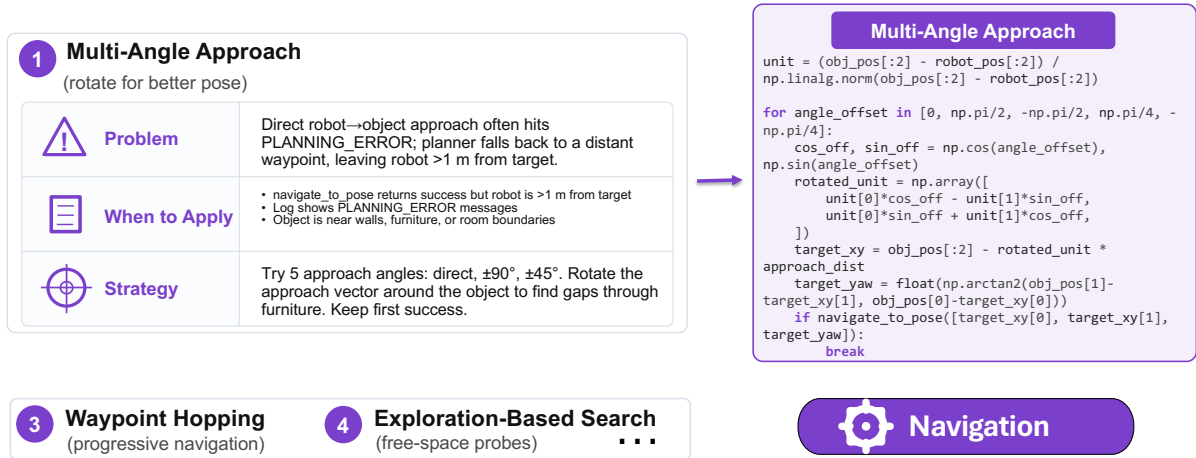


图 9: 导航技能。用于从运动规划失败中恢复并选择碰撞感知接近姿态的代表性条目。

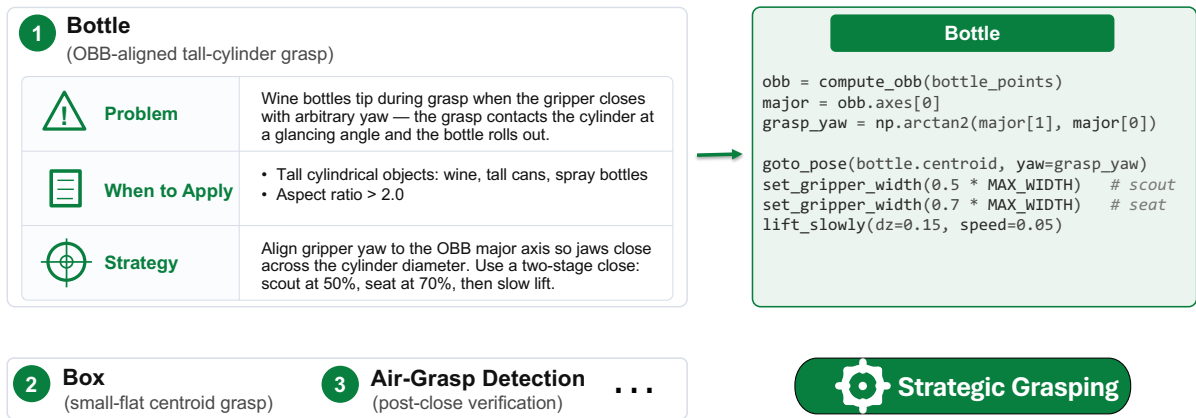


图 10: 策略性抓取技能。用于选择适合任务的抓取点，并根据物体几何形状与场景上下文调整抓取策略的代表性条目。

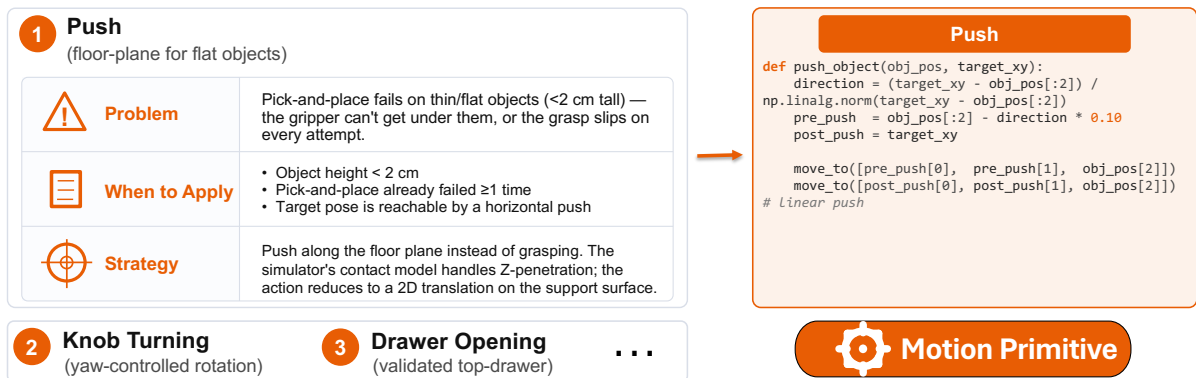


图 11: 运动基元技能。关于可复用低层运动模式、接触丰富的对齐以及执行时恢复的代表性条目。

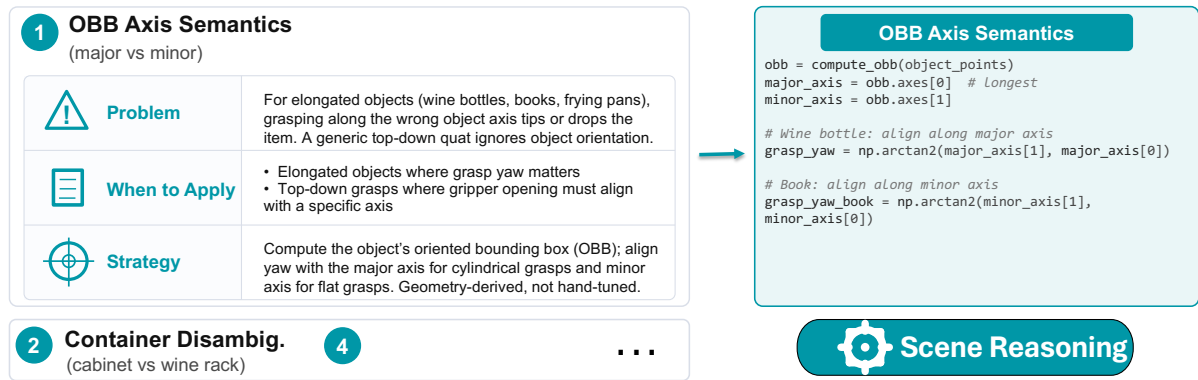


图 12: 场景推理技能。用于推理空间关系、支撑表面、遮挡以及场景级任务约束的代表性条目。

B. 主基准结果表

B.1. 宏平均主结果

本节报告主文图 4 中可视化的精确数值。

表 2: OpenVLA [Kim et al., 2024]、 π_0 [Black et al., 2025]、 $\pi_{0.5}$ [Physical Intelligence et al., 2025]、CaP-Agent0 [Fu et al., 2026] 与 ASPIRE 在 LIBERO-Pro [Zhou et al., 2025] 的 **libero-object**、**libero-goal** 和 **libero-spatial** 基准上的性能，评估条件包括初始位置扰动（位置）和指令扰动（任务），并在每个套件的 10 个任务上进行宏平均。**总体**列是在三个套件上进行宏平均。所有 ASPIRE 数值均来自 50 个种子的留出评估（seeds 1–50）。逐任务分解见附录 D.1。

方法	libero-object		libero-goal		libero-spatial		总体		
	位置	任务	位置	任务	位置	任务	位置	任务	全部
OpenVLA	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
π_0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
$\pi_{0.5}$	0.17	0.01	0.38	0.00	0.20	0.01	0.25	0.01	0.13
CaP-Agent0	0.22	0.18	0.26	0.17	0.12	0.14	0.20	0.16	0.18
Aspire	0.98	0.95	0.81	0.45	0.51	0.60	0.77	0.67	0.72

表 3: CaP-Agent0 [Fu et al., 2026] 与 ASPIRE 在 Robosuite [Zhu et al., 2020] 的 7 个操作任务上的性能（100 次试验的留出评估）。数值为 $[0, 1]$ 范围内的任务成功率。**平均**是全部 7 个任务的非加权平均。

任务	CaP-Agent0	Aspire
cube_lift	0.97	0.97
cube_stack	0.98	0.99
cube_restack	0.89	1.00
spill_wipe	1.00	0.99
two_arm_handover	0.20	0.92
two_arm_lift	0.74	0.71
nut_assembly	0.00	0.09
平均	<i>0.68</i>	<i>0.81</i>

表 4: BEHAVIOR-1K [Li et al., 2024] 在两个代表性家庭任务上的性能：汽水罐拾取和收音机拾取。分别报告 $[0, 1]$ 范围内的导航成功率和任务成功率；所有数值均来自 25 个种子的留出评估（seeds 1–25）；ASPIRE 使用在 seeds 26–35 上积累的技能库进行交互式逐块生成。

任务	人类		CaP-Agent0		Aspire	
	导航	任务	导航	任务	导航	任务
汽水罐拾取	0.80	0.72	0.84	0.72	0.92	0.88
收音机拾取	0.88	0.36	0.80	0.56	1.00	0.88

C. LIBERO-Pro Long 零样本迁移

C.1. 宏平均主结果

表 5: ASPIRE 技能库（在 LIBERO-90 上积累）到 LIBERO-Pro Long [Zhou et al., 2025] 的宏平均零样本迁移。 N 表示其修复技能用于初始化技能库的 LIBERO-90 任务数量；测试时不执行额外调试。各列报告位置（位置扰动）和任务（指令扰动）两个轴上的宏平均成功率，每个轴 10 个任务。

方法	LIBERO-Pro Long 零样本迁移		
	位置	任务	总体
OpenVLA	0.00	0.00	0.00
π_0	0.00	0.00	0.00
$\pi_{0.5}$	0.08	0.01	0.05
CaP-Agent0	0.052	0.024	0.038
ASPIRE ($N=0$, 零样本)	0.00	0.094	0.047
ASPIRE ($N=25$, 零样本)	0.056	0.218	0.137
ASPIRE ($N=50$, 零样本)	0.138	0.292	0.215
ASPIRE ($N=90$, 零样本)	0.226	0.383	0.305

C.2. 逐任务分解

表 6: ASPIRE 技能库（在 LIBERO-90 上积累）到 LIBERO-Pro Long 的零样本迁移，按任务和光照大小 N 分解（seeds 1–50, $n=50$ ）。**位置**：位置扰动变体。**任务**：语义扰动变体。

任务	位置				任务			
	$N=0$	$N=25$	$N=50$	$N=90$	$N=0$	$N=25$	$N=50$	$N=90$
炉灶 + 摩卡壶	0.00	0.30	0.84	1.00	0.00	0.02	0.68	0.26
碗在抽屉中	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
马克杯在微波炉中	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
两个摩卡壶在炉灶上	0.00	0.00	0.00	0.00	0.00	0.00	0.84	1.00
汤 + 奶油奶酪	0.00	0.12	0.02	0.00	0.00	0.68	0.60	0.70
汤 + 番茄酱汁	0.00	0.14	0.20	0.00	0.78	0.68	0.02	0.70
奶油奶酪 + 黄油	0.00	0.00	0.06	0.94	0.00	0.00	0.34	0.86
马克杯在两个盘子上	0.00	0.00	0.16	0.32	0.16	0.28	0.08	0.00
马克杯 + 布丁	0.00	0.00	0.06	0.00	0.00	0.00	0.08	0.04
书在置物盒中	0.00	0.00	0.04	0.00	0.00	0.52	0.28	0.26
平均	<i>0.00</i>	<i>0.06</i>	<i>0.14</i>	<i>0.23</i>	<i>0.09</i>	<i>0.22</i>	<i>0.29</i>	<i>0.38</i>

D. 消融实验详情

D.1. LIBERO-Pro 位置扰动 (Swap) 逐任务分解

表 7: LIBERO-Pro 逐任务消融，位置扰动 (seeds 1–50, $n=50$)。Aspire 去除机器人执行引擎和演化搜索：使用 15 个示例程序的零样本 Claude Opus 4.6。Aspire 去除演化搜索：机器人执行引擎修复后的程序 (执行引擎 + 技能库)。演化搜索：原始演化搜索结果 (最佳已验证候选)，如果该任务未运行演化搜索，则为 “–”。Aspire：在修复后程序和演化搜索之间，根据留出验证集 (seeds 66–80) 选择的逐任务胜者。

任务	无引擎 & 演化	无演化	演化搜索	Aspire
<i>libero-object</i>				
字母汤	0.00	1.00	–	1.00
烧烤酱	0.06	1.00	–	1.00
黄油	0.00	1.00	–	1.00
巧克力布丁	0.00	1.00	–	1.00
奶油奶酪	0.92	1.00	–	1.00
番茄酱	0.02	0.92	–	0.92
牛奶	0.06	0.92	–	0.92
橙汁	0.46	1.00	–	1.00
沙拉酱	0.18	0.96	–	0.96
番茄酱汁	0.66	0.98	–	0.98
平均	<i>0.24</i>	<i>0.98</i>	–	<i>0.98</i>
<i>libero-goal</i>				
打开中间抽屉	0.28	0.02	0.72	0.72
打开上层抽屉 + 碗	0.00	0.86	–	0.86
将盘子推到炉灶	0.00	0.00	0.00	0.00
碗放在盘子上	0.04	0.36	0.92	0.92
碗放在炉灶上	0.16	0.56	0.96	0.96
碗放在柜台面上	0.72	0.74	1.00	1.00
奶油奶酪放入碗中	0.00	0.74	0.92	0.74
葡萄酒放在架子上	0.06	0.46	0.98	0.98
葡萄酒放在柜台面上	1.00	0.24	1.00	1.00
打开炉灶	0.40	0.68	0.90	0.90
平均	<i>0.27</i>	<i>0.47</i>	<i>0.82⁽ⁿ⁼⁹⁾</i>	<i>0.81</i>
<i>libero-spatial</i>				
盘子 + 小烤盅之间	0.02	0.10	0.94	0.94
从桌面中心	0.00	0.32	0.82	0.32
在上层抽屉中	0.02	0.22	0.10	0.10
在饼干盒旁	0.02	0.16	0.12	0.16
在盘子旁	0.08	1.00	1.00	1.00
在小烤盅旁	0.00	0.20	0.40	0.20
在饼干盒上	0.00	0.32	0.24	0.24
在小烤盅上	0.58	0.42	0.94	0.94
在炉灶上	0.04	0.86	–	0.86
在木柜上	0.10	0.56	0.34	0.34
平均	<i>0.09</i>	<i>0.42</i>	<i>0.54⁽ⁿ⁼⁹⁾</i>	<i>0.51</i>
总体平均	<i>0.20</i>	<i>0.62</i>	<i>0.68⁽ⁿ⁼¹⁸⁾</i>	<i>0.77</i>

D.2. LIBERO-Pro 任务扰动 (Goal) 逐任务分解

表 8: LIBERO-Pro 逐任务消融, 任务扰动 (seeds 1–50, $n=50$)。Aspire 去除机器人执行引擎和演化搜索: 使用 15 个示例程序的零样本 Claude Opus 4.6。Aspire 去除演化搜索: 机器人执行引擎修复后的程序 (执行引擎 + 技能库)。演化搜索: 原始演化搜索结果 (最佳已验证候选), 如果该任务未运行演化搜索, 则为 “–”。Aspire: 在修复后程序和演化搜索之间, 根据留出验证集 (seeds 66–80) 选择的逐任务胜者。

任务	无引擎 & 演化	无演化	演化搜索	Aspire
<i>libero-object</i>				
字母汤	0.00	1.00	–	1.00
烧烤酱	0.88	1.00	–	1.00
黄油	0.78	0.98	–	0.98
巧克力布丁	0.00	1.00	–	1.00
奶油奶酪	0.00	0.72	0.74	0.72
番茄酱	0.00	0.96	–	0.96
牛奶	0.00	1.00	–	1.00
橙汁	0.00	1.00	–	1.00
沙拉酱	0.00	0.92	–	0.92
番茄酱汁	0.04	0.96	–	0.96
平均	0.17	0.95	–	0.95
<i>libero-goal</i>				
打开中间抽屉	0.00	0.00	0.00	0.00
打开上层抽屉 + 碗	0.00	0.50	0.46	0.46
将盘子推到炉灶	0.26	0.58	0.76	0.76
碗放在盘子上	0.00	0.88	0.82	0.88
碗放在炉灶上	0.00	0.00	0.00	0.00
碗放在柜台面上	0.00	0.00	0.04	0.04
奶油奶酪放入碗中	0.00	0.82	0.74	0.74
葡萄酒放在架子上	0.00	0.74	0.86	0.86
葡萄酒放在柜台面上	0.00	0.14	0.72	0.14
打开炉灶	0.24	0.52	0.66	0.66
平均	0.05	0.42	0.51 ^($n=10$)	0.45
<i>libero-spatial</i>				
盘子 + 小烤盅之间	0.14	0.08	0.84	0.08
从桌面中心	0.26	0.46	0.64	0.64
在上层抽屉中	0.04	0.96	–	0.96
在饼干盒旁	0.00	0.94	–	0.94
在盘子旁	0.08	0.70	0.92	0.70
在小烤盅旁	0.00	0.60	0.42	0.60
在饼干盒上	0.00	0.06	0.70	0.70
在小烤盅上	0.20	0.10	0.46	0.46
在炉灶上	0.00	0.32	0.42	0.42
在木柜上	0.13	0.46	0.98	0.46
平均	0.04	0.47	0.67 ^($n=8$)	0.60
总体平均	0.09	0.61	0.59 ^($n=19$)	0.67

D.3. 演化搜索进展逐任务分解

表 9: 选定 LIBERO-Pro 任务上的演化搜索进展。每一列显示该迭代中最佳候选的留出成功率 (seeds 1–50)。空白单元表示搜索提前终止。

套件	任务	迭代 0	迭代 1	迭代 2	迭代 3	迭代 4
goal_swap	碗 → 盘子	0.62	0.60	0.60	0.18	0.86
goal_swap	葡萄酒瓶 → 架子	0.40	0.76	0.74		
goal_swap	碗 → 炉灶	0.62	0.82			
goal_task	推盘子 → 炉灶	0.00	0.80			
spatial_swap	柜子上的碗 → 盘子	0.16	0.38	0.38	0.46	0.30
spatial_swap	饼干旁的碗 → 盘子	0.16	0.20	0.04	0.04	0.40
spatial_task	饼干上的碗 → 盘子	0.26	0.04	0.70		
spatial_task	小烤盅上的碗 → 盘子	0.02	0.00	0.22	0.36	0.18

E. 智能体流水线技能

E.1. 技能库提示词

执行者智能体使用结构化的 findings 模式报告候选修复：观察到的失败模式、已验证的修复、可迁移模式、任务特定的异常点，以及在调试配置上的验证成功率。随后，协调器提示词会审计每个候选选项的可复用性，检查 API 策略合规性，并且只接纳那些已验证且可能迁移到源任务之外的修复。并行执行者智能体被要求写入任务级修复和 findings，而协调器会串行化技能接纳流程，以避免对技能库的写入发生冲突。确切格式见 E.3 和 E.4 中的子智能体技能。所有智能体技能文件见代码发布包。

E.2. 智能体系统提示词与持久记忆

CLAUDE.md

```
# CLAUDE.md
This file is the constitution for Claude Code working in this repository. Read it fully before doing anything.

## STARTUP: Read Project Memory First

At the start of every session, immediately read `.claude/memory/MEMORY.md` before doing anything else.
It contains project vision, key conventions, and the skills index.

## What Is ASPIRE

ASPIRE is a framework where LLMs write Python code to control robots through a curated tool API. Code is
executed in a sandboxed simulator, the simulator verifies success, and the agent improves through
failure diagnosis and skill accumulation.

## FORBIDDEN APIs

These APIs access simulator ground truth and are STRICTLY FORBIDDEN in all fix code, debug scripts, and
skill implementations. Using them invalidates benchmark results, as they don't transfer to real
robots.

...

[FORBIDDEN] env.handle.env.sim - no MuJoCo sim object access
[FORBIDDEN] sim.data.body_xpos - no ground-truth object positions
... (rest of APIs omitted)
...

Rule of thumb: If a real robot with a camera could do it, it's allowed. If it reads the physics engine
's internal state, it's forbidden.

Also forbidden: Reading simulator asset files (`.bddl`, `.xml`, `.urdf`, MuJoCo model files) to infer
object geometry, success predicates, or scene structure. Treat these as inaccessible - diagnose
purely from observations and traces.

## ALLOWED APIs

Full source: integrations/franka/libero_reduced_skill_library.py` (and libero_reduced.py` for base class
). Read the source for exact signatures, return types, and edge cases.
...

[ALLOWED] get_observation() - RGB, depth, intrinsics, extrinsics, robot state
[ALLOWED] segment_sam3_text_prompt(rgb, text) - SAM3 text-prompted segmentation
... (rest of APIs omitted)
...

You are encouraged to design new helper functions (e.g. localize_object()`, make_topdown_quat()`)
when you find yourself repeating patterns. Add them to the relevant skill in `.claude/skills/` so
future sessions can reuse them.

## Skills
```


All skills live in ``.claude/skills/<name>/SKILL.md`` and are auto-discovered. Read the relevant skill before starting work.

```
| Skill | What it covers |
|---|---|
... (skills list omitted)
| **add more as needed** | **document new patterns here as you discover them** |
```

Log Everything

Append to ``./docs/logs/YYYY-MM-DD.md`` after any significant work: experiments run, fixes tried, skills updated, results observed. Be detailed - include trial numbers, success rates, error messages, file paths.

MEMORY.md

ASPIRE Project Memory

What We're Building

A Self-Improving Coding Agent for robot manipulation. The agent debugs its own failures, accumulates reusable skills, and improves its perception tools.

- Foundation: Rich debugging playground (traces, REPL)
- Layer 1: Auto-growing skill library
- Layer 2: Program search (Pass@K parallel search)

Key Conventions

- **NEVER git commit unless user explicitly asks**, do NOT auto-commit after skill updates or log writes
- **NEVER read sim asset files** (``.bddl``, ``.xml``, ``.urdf``, MuJoCo models), diagnose from observations and traces only
- **Always write reusable analysis as scripts** (not inline ``python3 -c "...``") and reference them in the relevant skill
 - ... (rest of conventions omitted)

Skills Available

... (skills list omitted)

api-reference.md

Observation and Perception

```
| Function | Returns |
|---|---|
| `get_observation()` | RGB image, depth image, camera intrinsics, camera pose, and robot state. |
| `segment_text_prompt(rgb, text_prompt)` | A list of masks with bounding boxes, scores, and labels. |
| `segment_point_prompt(rgb, point_coords)` | A list of masks from point-conditioned segmentation. |
| `point_prompt_vlm(image, text_prompt)` | A pixel coordinate for the prompted object or region. |
```

Grasp Planning and IK

```
| Function | Returns |
|---|---|
| `plan_grasp(depth, intrinsics, segmentation)` | Candidate grasp poses and scores in the camera frame. |
| `select_top_down_grasp(poses, scores, ...)` | The best top-down grasp transform and score. |
| `solve_ik(position, quaternion_wxyz)` | A 7-DoF joint configuration, or failure if unreachable. |
```

Camera-to-world conversion is required after grasp planning.

```
```python
camera_to_world = obs["agentview"]["pose_mat"]
grasp_world = camera_to_world @ grasp_camera
```
```

Motion Execution

| Function | Returns |
|--------------------------------------|---|
| `move_to_joints(joints)` | Blocking joint-space motion. |
| `open_gripper()` / `close_gripper()` | Gripper command. |
| `goto_home_joint_position()` | Move to a predefined safe home configuration. |

Geometry Utilities

| Function | Description |
|---|---|
| `decompose_transform(T)` | Convert a 4x4 transform into position and quaternion. |
| `mask_to_world_points(mask, depth, intrinsics, extrinsics)` | Project a 2D mask into a world-frame point cloud. |
| ... | (rest omitted) |

Task Language

```
```python
env.task_language
```
```

The runtime task language is the reliable source of the actual goal, especially when benchmark filenames do not match remapped task instructions.

Success Conditions

| Placement Type | Tolerance |
|--------------------------|----------------------------------|
| Regular object placement | Tight XY tolerance plus contact. |
| Site-object placement | Larger site-local tolerance. |

Scene Coordinate Frames

Use observation-derived depth projection for object positions and avoid hard-coded scene coordinates. Reference frame constants are environment-specific and should not be used as object localization shortcuts.

`... (benchmark-specific table of scene constants omitted) ...`

Output Directory Structure

```
```text
<OUTPUT_DIR>/
 <suite>/<task>/<run_id>/
 run/
 <done_flag>
 trial_<seed>_<attempt>_reward_<score>_taskcompleted_<flag>/
 code.py
 trace.json
 keyframes/
 video_combined.mp4
 visual_feedback.png
 summary.txt
 all_responses.json
```
```

The trial folder name records the seed, attempt index, reward, and task-completion flag. The attempt index distinguishes clean exits from crashes, and `reward` is the scalar task score.

Trace Format

```
```json
{
 "step": 5,
 "function": "segment_text_prompt",
 "args": {"rgb_shape": [512, 800, 3], "text_prompt": "bowl"},
}
```

```

 "duration_ms": 1463.8,
 "result": {
 "num_masks": 200,
 "mask_0_score": 0.917,
 "mask_0_bbox": [369.2, 257.1, 438.4, 310.5],
 "mask_0_area_pct": 0.68
 },
 "keyframe_saved": true
 }
}
...

Key Source Files

| File | Role |
|---|---|
| `<batch_runner>` | Runs suites, tasks, and model configurations. |
| `<trial_loop>` | Resets the environment, executes generated code, and saves artifacts. |
| `<single_trial_runner>` | Replays saved code or runs a fresh trial. |
| `<single_trial_analyzer>` | Performs structured trace analysis for one trial. |
| `<multi_candidate_trace_analyzer>` | Performs per-candidate trace analysis for evolutionary search runs. |
| `<progress_scanner>` | Aggregates success rates across an output directory. |
| `<robot_api_source>` | Defines the robot control and geometry API. |
| `<trace_logger_source>` | Records API calls, return values, timings, and keyframes. |
| `<evaluation_config>` | Defines benchmark and replay configuration. |

```

### E.3. ASPIRE 修复循环技能

coordinator.md

```

> **Purpose:** Coordinate repair of failed robot trials from a baseline policy and validate finalized
 fixes on a held-out seed set.
> **Baseline results:** `<BASELINE_OUTPUT_DIR>/`
> **Progress tracker:** `<PROGRESS_FILE>`
> **Subagent prompt:** `<SUBAGENT_PROMPT_TEMPLATE>`

Initialization: Verify Perception Services

Before dispatching subagents, confirm that all required perception services are running. A service-
specific health check should return the expected "up" status.

If any service is down, start it before proceeding. Subagents may fail silently without these dependencies
.

Main Loop

```text
read progress → assign free devices → dispatch subagents → go idle
on notification: update progress/skills + redispach device --|
```

One task is handled by one subagent on one compute device. Repeat until no eligible tasks remain.

Coordinator Rules

1. **Dispatch subagents rather than debugging directly.** The coordinator assigns work, waits for
 completion notifications, updates shared knowledge, and dispatches the next task.
2. **Go idle after dispatching.** Completion notifications indicate when a device is free; avoid polling
 or intervening while subagents are running.
3. **Keep available devices occupied.** Any free device should receive the next pending task.
4. **Do not inspect task-specific debugging artifacts.** Files such as traces, generated programs,
 summaries, or keyframes are the subagent's responsibility. The coordinator may read the subagent's
 final findings when updating shared skills.

```

```

5. **Never re-dispatch completed tasks.** Completed validation tasks must not be run again, because
 duplicate trial directories can corrupt aggregate results.

Workflow

1. Read the Progress Tracker

```bash
cat <PROGRESS_FILE>
```

Identify tasks marked `pending` or `stage1-complete`.

2. Check Free Compute Devices

Assign tasks only to devices that are free. Never assign two tasks to the same device.

3. Dispatch One Subagent per Free Device

```python
Agent(
    description="Fix loop: <suite>/<task>",
    subagent_type="general-purpose",
    prompt=<filled_subagent_prompt>,
    run_in_background=True,
)
```

Send all dispatches together, then stop. The coordinator resumes when completion notifications arrive.

4. On Completion: Redispatch and Update Skills

When a subagent completes, its result should include the device identifier. The coordinator then:

1. Reads the progress tracker.
2. Dispatches the next eligible task on the freed device.
3. Reads the completed task's `findings.md`.
4. Promotes generalizable patterns into the shared skill library.
5. Goes idle again.

If the result does not include a device identifier, query the device monitor to infer which device is free
.

5. Update Shared Skills

After each completion, read:

```text
<BASELINE_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md
```

Promote only generalizable patterns to the skill library.

`... (skill-library category table omitted) ...`

Validation Runs

For pure validation runs where final repair code already exists, direct scripts are more reliable than
autonomous subagents. The validation script should run the held-out seed set exactly once and write
to the official output directory.

`... (task-specific validation script, log parsing, and environment variables omitted) ...`

Important rule: pass only the validation base directory if the replay script appends `<suite>/<task>`
internally.

Suite Name Collision Warning

```

Some benchmark suites may contain identical task names under different perturbation conditions. Always include the full suite name when briefing a subagent.

### ### Perturbation Notes

- Position-perturbation suites randomize object poses by seed; perception should handle this naturally.
- Language-perturbation suites remap the task goal; the executable task instruction should be read from the runtime environment rather than inferred from a filename.

## subagent.md

The following prompt is filled with one suite, task, and device assignment before being passed to a background subagent.

```
```text
```

Task Assignment

SUITE: <SUITE_NAME>

TASK: <TASK_NAME>

DEVICE: <DEVICE_ID>

Working directory: <PROJECT_ROOT>

```
```
```

### ### Evaluation-Set Lockout

The held-out validation seeds are locked during debugging.

- During Stage 1, replay only the debugging seeds.
- Do not replay validation seeds for testing, tuning, temporary experiments, or alternate code versions.
- Stage 2 uses the validation seeds exactly once, after `fix\_code.py` is finalized, and writes to the official validation output directory.
- Stage 2 must run to completion before the subagent returns.

Violating this rule invalidates the benchmark result.

### ### Role

You are a task-level repair subagent for a robotics benchmark. Your job is to diagnose failed robot trials for one task, write a generalizable repair program, validate it according to the protocol, and report concise results.

### ### Context

The system asks language models to write Python control programs for a robot arm through a curated perception and manipulation API. Programs run in simulation, and failures are diagnosed through traces, summaries, and visual keyframes. The baseline policy has already run a set of debugging seeds ; your job is to repair the task program without using simulator ground truth.

### #### Forbidden APIs

The repair program must not access simulator internals, ground-truth object states, hidden task predicates , asset files, or low-level physics state. Examples include:

```
```text
```

```
simulator body positions
```

```
site positions
```

```
`... (additional forbidden APIs omitted) ...`
```

```
```
```

These APIs would not transfer to a real robot and invalidate the result.

### #### Allowed APIs

The repair code may use only observation-derived perception, manipulation primitives, and standard numerical computation:

```
```text
get_observation()
segment_text_prompt(rgb, text)
`... (additional allowed APIs omitted) ...`
```
```

Always use the project virtual environment or pinned runtime, not the system Python.

Before any replay, verify that the required perception services are running.

### ### Stage 1: Debug on Non-Held-Out Seeds

#### #### Step 0: Check Whether Repair Code Already Exists

If it exists, skip Stage 1 and check whether Stage 2 has already completed.

#### #### Step 1: Fast Path from Successful Baseline Trials

First, check whether any baseline debugging seed already succeeded:

If successful trials exist:

1. Read the successful generated program.
2. Extract the final code block if multiple attempts are present.
3. Test it on a small number of failed debugging seeds.
4. If it succeeds, save it as the task-level `fix\_code.py` and proceed to Stage 2.
5. If it fails, continue to the full debug loop.

If no debugging seeds succeeded, continue directly to the full debug loop.

```
`... (exact replay command and temporary output paths omitted) ...`
```

#### #### Step 2: Diagnose Failed Debugging Seeds

For each failed seed, inspect the last attempt for that seed:

- `trace.json`: function calls, return values, perception failures, IK failures, and gripper signals.
- `code.py`: the baseline strategy.
- `keyframes/`: visual observations at perception steps.
- `summary.txt`: stdout, stderr, and reward.

Interpretation examples:

```
```text
large gripper width after closing → likely object grasped
medium gripper width → marginal grasp
small gripper width → likely air grasp
zero masks → poor perception prompt or occlusion
missing IK solution → target outside reachable workspace
early crash → inspect stdout/stderr summary
```
```

#### #### Step 3: Write and Test Repair Code

Before writing code, read the relevant skill-library entries for localization, grasping, transport, and manipulation patterns.

Write `fix.py` from the diagnosis and replay it on debugging seeds only.

If the repair fails, read the new trace and iterate. A live inspection REPL may be used for observation-derived checks such as segmentation counts.

```
`... (interactive REPL command and shell-specific buffering notes omitted) ...`
```

Hard limit: at most three replay attempts per debugging seed. Any run against a seed counts as an attempt, regardless of output directory or code version. After the cap, write a `BLOCKED.md` report and move

```

on.

`BLOCKED.md` format:

`... (blocked-seed report template omitted) ...`

Step 4: Synthesize the Task-Level Repair Program

After debugging all seeds, synthesize one generalizable `fix_code.py`. The code must not contain seed-
specific branches.

Save it to `///fix_code.py`. Also write `///findings.md`.

Use this findings format:

```markdown
## Task: <SUITE> / <TASK>

### Root Cause(s)
- <concise description of each failure mode>

### What Fixed It
- <change that resolved each root cause>

### Perception Prompts That Worked
| Object | Prompts (priority order) | Notes |
|---|---|---|
| <object> | "<prompt1>", "<prompt2>" | <caveats> |

### Generalizable Patterns
- <patterns likely to apply beyond this task>

### Task-Specific Quirks
- <details specific to this task>

### Stage 2 Success Rate
<N>/<TOTAL_VALIDATION_SEEDS>
```

If all debugging seeds are blocked, skip Stage 2 for this task.

Stage 2: Validate on Held-Out Seeds

Stage 2 is a one-shot validation operation. Do not debug Stage 2 runs.

Before running, check whether validation has already completed:

If enough unique validation seeds are already present, do not re-run Stage 2. Report the existing results
and return. Otherwise, run validation sequentially:

`... (validation loop, log parsing, success counting, and environment variables omitted) ...`

Verify that outputs landed in the expected directory before reporting results.

Final Action

Run the shared progress-generation script as the last action. It should update the progress tracker
atomically and be safe under concurrent subagent execution.

Return Format

Return only a brief structured summary:

```text
SUITE: <SUITE>
TASK: <TASK>
DEVICE: <DEVICE>

```



```

Stage 1: fix_code.py written: yes/no
  Seeds fixed: <count or list>
  Seeds blocked: <count and one-line cause>

Stage 2: <N>/<TOTAL_VALIDATION_SEEDS>
  Output dir: <VALIDATION_OUTPUT_BASE>/<SUITE>/<TASK>/

Key findings:
- <generalizable root cause and fix>
- <useful perception prompt, if notable>
- <pattern that should inform future tasks>
...

```

E.4. ASPIRE 演化搜索技能

演化搜索（第 2.3 节）维护一个跨轮次携带的持久任务分析文档 A_i 。 A_i 由以下部分组成：(i) 从初始快照一次性填充的逐任务场景描述（物体形状、目标几何、障碍物、被阻塞的接近方向），(ii) 持续更新的假设以及用于检验这些假设的候选元数据，(iii) 一份记录已排除方向（通过测试排除）和被阻塞、未测试方向（因工作空间约束而失败）的台账。UPDATEANALYSIS 每一轮都会基于新的轨迹、关键帧和库检索结果重写 A_i 。这个文档使后续轮次能够避免重新测试已排除的分支，同时在新技术（例如腕部旋转）可用后仍能重试被阻塞的分支。

coordinator.md

```

> What: Run Evolutionary search iterative debugging with multiple candidates per iteration on a list
  of low-performing tasks.
> Why: Intensive debugging for difficult tasks; multi-candidate search can find strategies the single-
  pass actor misses.
> Subagent template: `<EVOSEARCH_SUBAGENT_PROMPT_TEMPLATE>`

### Initialization: Verify Perception Services

Before dispatching subagents, verify that all required perception services are running.
All services must be up before dispatching.

`... (service health-check command and machine-specific ports omitted) ...`

### Directory Layout

```

`text
<EVOSEARCH_OUTPUT_DIR>/
 <suite>/<task>/<run_id>/
 task_analysis.md
 iter_00/ iter_01/ ...
 candidate_A/code.py
 iter_summary.json
 <suite>/<task>/
 evosearch_best_code.py
 findings.md

<VALIDATION_OUTPUT_DIR>/
 <suite>/<task>/... (rest omitted)
...

```

### Task List

Run `<progress_generation_script>` to verify exact task names before dispatch.
Tasks below the chosen baseline success threshold are candidates for repair.

`... (experiment-specific task ordering and threshold table omitted) ...`

```

The Loop

```
```text
read progress → assign free devices → dispatch subagents → go idle
 ^
on notification: redispatch freed device to next pending task -|
```
```

One task = one subagent = one compute device.

The subagent runs iterations on the debug set and then runs Stage 2 on the held-out set before returning. When the coordinator gets a completion notification, the device is already free; dispatch the next task.

Coordinator Rules

1. **Dispatch subagents** -- never run iterations yourself.
2. **Go idle** after dispatching. You will be notified when a subagent finishes.
3. **Keep available devices occupied.**
4. **On each notification:** check which device freed up, dispatch next pending task to it.
5. **Never re-dispatch a done task.** `done` means Stage 2 complete with enough unique held-out seeds on disk.

Workflow

1. Check Progress

Check which tasks have a saved `evosearch\_best\_code.py` and how many held-out seeds have already been evaluated.

`... (progress-check shell loop omitted) ...`

2. Check Free Compute Devices

Assign tasks only to devices that show no active jobs.

`... (device-monitoring command omitted) ...`

3. Dispatch Subagents

Use the stronger reasoning model selected for the experiment.

```
```python
Agent(
 description="Evolutionary search actor: <suite_short>/<task_short> device<N>",
 subagent_type="general-purpose",
 model="<HIGH_CAPACITY_MODEL>",
 prompt=<filled_evosearch_subagent_prompt>,
 run_in_background=True,
)
```
```

Send all dispatches in one message, one per free device, then stop.

4. On Each Completion: Redispatch

When a subagent notification arrives, it has already completed both Stage 1 and Stage 2.

Read ``<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md``, note Stage 1 and Stage 2 rates, check free devices, dispatch the next pending task, and go idle.

Stopping Criteria Reference

Subagents stop Stage 1 when the best candidate reaches the success threshold on debug seeds, the maximum number of iterations is reached, or the task is blocked.

If Stage 1 is blocked, the subagent skips Stage 2 and returns immediately.

subagent.md

The following prompt is filled with one suite, task, device assignment, baseline rate, and output location before dispatch.

```

```text
Task Assignment

SUITE: <SUITE>
TASK: <TASK>
DEVICE: <DEVICE_ID>
TASKSHORT: <SHORT_UNIQUE_NAME_FOR_LOGS>
BASELINE_RATE: <BASELINE_RATE>
EVOSEARCH_DIR: <EVOSEARCH_OUTPUT_DIR>
EVALDIR: <VALIDATION_OUTPUT_DIR>

Working directory: <PROJECT_ROOT>
```

### Evaluation-Set Lockout During Iterations

The held-out evaluation seeds must not be run during iterations.
All iteration evaluations use only the debug seed set.
Stage 2 uses the held-out seeds after iterations converge and the final code is chosen.
Violation invalidates the benchmark.

### Role

You are a debugging subagent.
Your job is to run evolutionary search iterative debugging on debug seeds until convergence or plateau,
save the best code, run Stage 2 on held-out seeds, and return a structured findings report with both
Stage 1 and Stage 2 results.
Use the project virtual environment or pinned runtime, never the system Python.

### Context

The actor pipeline achieved ``$BASELINE_RATE`` on this task.
Your target is to exceed that significantly, ideally reaching the Stage 1 success threshold on debug seeds
.
Read the skill library before writing candidates.
Verify that the required perception services are running before replay.

`... (experiment-specific config path and service-port checks omitted) ...`

### Forbidden APIs

```text
simulator body positions
site positions
`... (rest of forbidden APIs omitted)
```

### Stage 1: Evolutionary Search Iterations on Debug Seeds

#### Step 0: Check Whether Best Code Already Exists

If `evosearch_best_code.py` exists, skip to the return step.

#### Step 0b: Task-Language Remapping Check

The task name might not match the actual goal.
Always check and print the runtime task language.
If it differs from ``$TASK``, the runtime task language is ground truth; base all strategy on it and record
the actual language in `task_analysis.md`.

#### Step 1: Read Skills and Existing Baseline

```

Read skill-library entries for grasping, localization, transport, manipulation, and any task-specific companion skills.
 Also read existing repair code if it exists.
 If baseline code is found, record its path in `task\_analysis.md` and seed it as `candidate\_A` verbatim.

`... (skill-file reads and baseline fallback command omitted) ...`

Step 2: Create Run Directory and Scene Snapshot

Create a timestamped run directory under `///`.
 Before writing candidates, run a scene snapshot on one debug seed and inspect the wide and wrist-view images.
 Populate `task\_analysis.md` with object shape, grasp strategy, goal geometry, placement strategy, hypotheses for the first iteration, and an iteration log.

`... (snapshot command and task\_analysis.md template omitted) ...`

Step 3: Write K Candidates for the First Iteration

Always seed `candidate\_A` from the existing baseline repair code if it exists.
 Each candidate must test a distinct hypothesis.
 No two candidates should fail at the same stage for the same reason.
 Each candidate should include a docstring describing the hypothesis, how it differs from prior candidates, and the expected failure mode if wrong.

Step 4: Evaluate the Iteration

Evaluate all candidates on the fixed debug seed set.
 Use the same debug seeds across all iterations so cross-iteration comparison is fair.
 Read the leaderboard after completion and analyze traces.

`... (multi-candidate evaluation command, leaderboard parser, and trace analyzer command omitted) ...`

Step 5: Iterate

Stop Stage 1 when any stopping criterion is met.
 When progress stalls, use the remaining iteration budget to explore structurally new approaches rather than stopping early.
 A plateau on the current approach family means the current family may be wrong, not that the task is unsolvable.

The debug seeds are a small, potentially unrepresentative sample.
 The code will be evaluated on unseen held-out seeds; write for those, not for the debug seeds.
 Prefer strategies that work for mechanistic reasons over strategies that exploit patterns specific to debug-seed failures.
 Avoid hard-coded thresholds, image-region masks, or offsets derived by fitting to observed debug-seed failures.

Per iteration:

1. Update `task\_analysis.md` with new geometry from keyframes.
2. Append an iteration log with the leaderboard, eliminated hypotheses, and open questions.
3. Write K new candidates seeded from the top survivors of the current iteration.
4. Evaluate all candidates on the same debug seed set.
5. Read trace-analysis output and watch for arm blocking, gripper-width signals, perception failures, and IK failures.

Step 6: Save Best Code, Run Stage 2, Write Findings

Save the best final candidate as `///evosearch\_best\_code.py`.
 If the best candidate does not beat the baseline candidate on debug seeds, fall back to the stronger baseline so Stage 2 uses the best available code.
 If all approaches fail, mark the task blocked and skip Stage 2.

`... (best-candidate sanity check and copy commands omitted) ...`

Stage 2: Evaluate Best Code on Held-Out Seeds

```

Run the selected best code on each held-out seed exactly once.
Do not use Stage 2 results to revise the code.
Poll until the validation loop completes, then read final success counts.

`... (detached validation script, polling commands, and final count commands omitted) ...`

### Findings Report

Write `<EVOSEARCH_OUTPUT_DIR>/<SUITE>/<TASK>/findings.md`:

```markdown
Task: <SUITE> / <TASK>
Baseline rate: <BASELINE_RATE>
Best evosearch rate on debug seeds: <N>/<DEBUG_TOTAL> (<pct>%)
Stage 2 on held-out seeds: <N>/<HELD_OUT_TOTAL> (<pct>%)

Evolutionary Search Run
- Run dir: <RUN_DIR>
- Iterations completed: <N>
- Stopping reason: <solved|max_iterations|blocked>

What Fixed It vs Baseline
- <key strategy change>

Perception Prompts That Worked
| Object | Prompts | Notes |
|---|---|---|

Failure Modes Eliminated
- <approach that failed and why>

Generalizable Patterns
- <anything worth adding to the skill library>

Skill Library Updates Made
- <if any skills were updated during this run>
...

Return Format

```text
SUITE: <suite>
TASK: <task>
DEVICE: <N>
Baseline rate: <from actor pipeline>

Stage 1:
  Best candidate: <candidate_X> at iter_NN
  Best pass rate: <N>/<DEBUG_TOTAL> (<pct>%)
  Stopping reason: <solved|max_iterations|blocked>
  Iterations run: <N>
  Run dir: <RUN_DIR>

Stage 2: <N>/<HELD_OUT_TOTAL> (<pct>%)

Key findings:
- <strategy that worked vs baseline failure mode>
- <perception prompts or grasp parameters>
- <generalizable pattern for the skill library>
...

```

E.5. 初始技能库模板

grasp initial SKILL.md

```

---
name: grasp
description: Structural template for pick-and-place programs - topdown-quat construction, pre-grasp
  approach, lower-close sequence, lift-transport-place pattern. No task-specific parameters; those grow
  through experiment.
---

# Grasp - Structural Template for Pick-and-Place

> **Purpose:** Code skeletons that every pick-and-place task reuses. No task-specific z-offsets or yaw -
  those go in the per-task `task_code.py`.
> **Ownership:** Coordinator adds generalizable patterns. Subagents do not edit this file.

---

## Top-Down Quaternion

```python
import numpy as np
from scipy.spatial.transform import Rotation

def make_topdown_quat(yaw_deg=0):
 """Build a top-down end-effector quaternion (xyzw→wxyz convention used by solve_ik)."""
 R = Rotation.from_euler('z', yaw_deg, degrees=True).as_matrix() @ \
 np.array([[1, 0, 0], [0, -1, 0], [0, 0, -1]])
 q = Rotation.from_matrix(R).as_quat() # scipy returns xyzw
 return np.array([q[3], q[0], q[1], q[2]]) # reorder to wxyz
```

---

**Yaw selection:** default 0. If the grasp fails because the gripper fingers collide with a neighboring
object, try 90 or 45 degrees.

---

## Standard Pick-and-Place Skeleton

```python
Assume: obj_center, obj_mask, tgt_center, tgt_pts already localized (see localize skill)
Assume: depth, K (intrinsics), E (camera extrinsic pose_mat) from get_observation()

1. Plan grasp via GraspNet on object mask
2. Pre-grasp approach: gripper above object, open
3. Lower to grasp pose, close gripper
4. Lift (avoid dragging object sideways into neighbors)
5. Move above target (same z as lift)
6. Lower to release height (surface_z + margin)
7. Release
8. Let physics settle (important for drop-success reading)
... (skeleton code omitted)
```

---

## Why `select_top_down_grasp`?

`plan_grasp` returns grasps in camera frame, often angled or side-approach. We want top-down for table-top
pick-and-place. The selector transforms to world frame and filters for top-down direction (z-axis
roughly anti-aligned with gravity). When it returns None (no valid top-down), fall back to `E @
grasp_poses[grasp_scores.argmax()]`.

---

## When to NOT Use This Template

```

```

- **Shelves / drawers / microwaves** - use `manipulation` skill, not this.
- **Stacking** - need stacking-specific surface_z logic (stack on top of existing object, not table).
- **Narrow containers (baskets with walls)** - need yaw that avoids wall collision; release height above
  basket rim.

---

## Per-Object Grasp Registry (grows per chunk)

Discovered per-object grasp parameters. Coordinator adds entries from `findings.md`.

Object	z_offset	yaw	Notes	First Seen In

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object name | z_offset from OBB top | yaw_deg if non-zero | why | task that validated it -->

---

## Anti-Patterns

- `set_joint_qpos` to "teleport" the gripper into grasp pose - forbidden.
- Hardcoded grasp positions from task BDDL - forbidden (reading .bddl is banned).
- Lifting to fixed absolute z (e.g., z=0.9) - use `grasp_pos[2] + 0.15` to be scene-relative.
- Skipping the pre-grasp `z_approach=0.15` - causes gripper collisions with neighbors.
- Single-try `select_top_down_grasp` without `argmax` fallback - some scenes have no valid top-down grasp.

---

## Debugging Grasp Failures

Symptom	Likely Cause	Check
Gripper closes on air	z too high (didn't descend to surface)	Print `grasp_pos[2]` vs. `surface_z`;
use OBB `extent[2]/2` below top		
Grasp succeeds but drops during lift	Gripper half-closed on slippery/elongated object	Try
perpendicular yaw (90 degrees) or fingers-along-long-axis approach		
`plan_grasp` returns empty	Mask too small or below grasp threshold	Dilate mask; log `mask.sum()` -
should be >200 px		
`solve_ik` returns None	Pose out of workspace	Lower z by 5cm, re-check x/y reachable from robot base

```

localize initial SKILL.md

```

---
name: localize
description: Object localization via SAM3 - prompting strategies, multi-prompt fallback, 3D centroid
  extraction, disambiguation, per-object prompt registry. Grows through experiment.
---

# Localize - SAM3 Prompting & Object Localization

> **Purpose:** This skill tracks working SAM3 prompts per object + the standard localization helpers.
> **Ownership:** Coordinator adds entries to the prompt registry after reviewing subagent findings.
  Subagents do NOT edit this file.
---

## Standard Localization Helper

```python
import numpy as np

def localize_object(rgb, depth, K, E, prompts):
 """Try prompts in order, return (center, pts, mask) for first hit with >=10 points."""
 depth_img = depth[:, :, 0] if len(depth.shape) == 3 else depth
 if isinstance(prompts, str):

```

```

 prompts = [prompts]
 for prompt in prompts:
 masks = segment_sam3_text_prompt(rgb, prompt)
 if not masks:
 continue
 best = max(masks, key=lambda d: d["score"])
 mask = best["mask"].astype(np.uint8)
 pts = mask_to_world_points(mask, depth_img, K, E)
 if pts is None or len(pts) < 10:
 continue
 center = get_oriented_bounding_box_from_3d_points(pts)["center"]
 return center, pts, mask
 return None, None, None
...

Usage:
```python
obs = get_observation()
cam = obs["agentview"]
center, pts, mask = localize_object(
    cam["images"]["rgb"], cam["images"]["depth"],
    cam["intrinsic"], cam["pose_mat"],
    ["<specific prompt>", "<fallback prompt>"]
)
if center is None: raise RuntimeError("Object not found")
```

Why OBB center over mean: `get_oriented_bounding_box_from_3d_points` gives a more robust center for
elongated/occluded objects than `pts.mean(axis=0)`.

Fallback: Molmo Pixel Grounding

When SAM3 returns no masks even for sensible prompts, try Molmo to get a pixel, then convert with SAM3
point prompt:

```python
def localize_via_molmo(rgb, depth, K, E, phrase):
    px = point_prompt_molmo(rgb, phrase) # (x, y) or None
    if px is None: return None, None, None
    masks = segment_sam3_point_prompt(rgb, [px])
    if not masks: return None, None, None
    mask = masks[0]["mask"].astype(np.uint8)
    depth_img = depth[:, :, 0] if len(depth.shape) == 3 else depth
    pts = mask_to_world_points(mask, depth_img, K, E)
    if pts is None or len(pts) < 10: return None, None, None
    return get_oriented_bounding_box_from_3d_points(pts)["center"], pts, mask
...

---

## Disambiguation: Two Similar Objects in Scene

When a scene contains two visually similar objects, `max(score)` often picks the wrong one. Use `bbox
pixel area` or `bbox y-position` (upper = farther):

```python
def select_by_smallest_bbox(masks, max_cy=None, min_area=50):
 """Pick mask with smallest bbox pixel area (most compact matching shape).
 max_cy: if set, only accept masks with bbox center-y below this row."""
 ... (implementation omitted)
...

Key: `area_pct` in SAM3 result dicts clamps to 99.00 for all masks - compute area from `box` directly.

```



```

Disambiguation: By 3D Position (when two objects share prompts)

```python
# Example: "black bowl" returns both bowls, want the one at the front
... (implementation omitted)
```

Prompt Registry (grows per chunk)

Discovered working prompts, indexed by object. Coordinator appends entries after each chunk review.

Format per row: object | priority-ordered prompts | which task validated it | notes.

Object	Working Prompts	First Seen In	Notes

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object name | prompts in priority order (quoted) | first task that validated | disambiguation
 notes if needed -->

How to Add a New Prompt (Coordinator Only)

When a subagent's `findings.md` "Generalizable Patterns" section mentions a new working prompt:

1. **Verify generalizability** - would this prompt plausibly work for >=2 task types? If it's ultra-specific to one task, consider keeping it inline in `task_code.py` instead of here.
2. **Append to registry** - one table row, priority-ordered list of prompts.
3. **Add a disambiguation note** only if the task exposed a specific confusion (e.g., two bowls in frame).
4. **Keep examples short** - a full code block per prompt is overkill. The helper pattern stays at the top of this file; the registry is just prompt strings.

Anti-Patterns (Do NOT Do These)

- `body_xpos` for object localization - forbidden. If you're tempted, the perception is wrong; fix the prompt.
- Hardcoded pixel coordinates - always derive from SAM3 or Molmo.
- Caching object positions across seeds - re-localize every episode; objects are randomized.
- Using `segment_sam3_text_prompt` mask dict without `.astype(np.uint8)` conversion - silent failures downstream.

```

#### manipulation initial SKILL.md

```

name: manipulation
description: Non-pick-and-place manipulation patterns - drawer opening/closing, knob/switch turning,
 pushing, articulated-object interactions. Grows with validated parameters per object type.

Manipulation - Articulated Objects & Non-Prehensile Actions

> **Purpose:** Tracks validated parameters for articulated manipulation. Approaches are discovered through
 experiment and recorded in the registry below.
> **Ownership:** Coordinator adds validated parameters per object-type. Subagents do not edit.

Examples of Task Types Requiring This Skill

- **Drawer open/close** - pull/push a drawer by its handle
- **Knob/switch turn** - rotate a stove knob or flip a switch to trigger a state change
- **Hinged door** - open a microwave, cabinet door, or similar hinged panel

```

```

- **Non-prehensile push** - slide an object to a target without grasping it

These require non-top-down approach angles, arc motions, or contact-without-grasp patterns.

Parameter Registry (grows per chunk)

Validated parameters per object / action. Coordinator adds after verifying from `findings.md`.

Object / Action	Parameters	First Seen In	Notes

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->
<!-- Format: object/action | key parameters (distances, angles, offsets, approach direction) | first task
 | notes on what failed before -->

Anti-Patterns

- `set_joint_qpos` to force a drawer/door open - forbidden. Must use physical manipulation.
- Reading BDDL to get pull distances or knob angles - forbidden. Discover via OBB or trial-and-error.
- Forgetting to retreat after manipulation - arm stays inside drawer/microwave, blocks next perception.

```

#### transport initial SKILL.md

```

name: transport
description: Motion patterns for moving the end-effector between poses - multi-step waypoints, safe
 transit, interpolated Cartesian moves, collision avoidance. Grows with discovered patterns.

Transport - End-Effector Motion Between Poses

> **Purpose:** Transit patterns after a grasp is achieved and before a place/release. Keeps the object
 above clutter, avoids scene collisions, handles narrow workspaces.
> **Ownership:** Coordinator adds validated waypoint patterns. Subagents do not edit.

Basic Lift-Transit-Descend

```python
# Precondition: object grasped (close_gripper done), grasp_pos known, quat known
# Target: tgt_center (3D world coordinate), surface_z (target surface height)

# Waypoint 1: lift straight up (avoid dragging)
lift_z = grasp_pos[2] + 0.15
joints = solve_ik([grasp_pos[0], grasp_pos[1], lift_z], quat.tolist())
if joints is not None: move_to_joints(joints)

# Waypoint 2: move laterally at lift height (avoid hitting other objects on surface)
joints = solve_ik([tgt_center[0], tgt_center[1], lift_z], quat.tolist())
if joints is not None: move_to_joints(joints)

# Waypoint 3: descend to release height
release_z = surface_z + 0.05
joints = solve_ik([tgt_center[0], tgt_center[1], release_z], quat.tolist())
if joints is not None: move_to_joints(joints)
```

Why 3 waypoints, not direct goto_pose: a direct move from grasp to release tends to take a diagonal
path that clips tall objects, basket walls, or the drawer opening. Three cardinal waypoints (up →
over → down) keeps the object in free air during transit.

```

### ## Home Reset Between Subtasks

For long-horizon tasks (LIBERO-Long-Pro), reset the arm between subtasks to a known-safe config:

```
```python
def safe_reset_between_subtasks():
    open_gripper()
    goto_home_joint_position()
    for _ in range(3): get_observation() # let physics settle
```
```

**\*\*When to call:\*\*** after releasing subtask-1's object, before starting subtask-2's localization. Prevents subtask-2 from trying to perceive through the arm in frame.

---

### ## Far-Workspace Safety: `solve\_ik` over `goto\_pose`

`goto\_pose` is unreliable in the far workspace (e.g., back of counter, far side of table). Use `solve\_ik` + `move\_to\_joints` for these transits:

```
```python
# UNRELIABLE (far workspace):
# goto_pose(far_target_pos, quat)

# RELIABLE:
joints = solve_ik(far_target_pos.tolist(), quat.tolist())
if joints is None:
    # Try slightly closer z or x
    fallback = far_target_pos.copy(); fallback[2] -= 0.02
    joints = solve_ik(fallback.tolist(), quat.tolist())
if joints is not None:
    move_to_joints(joints)
else:
    raise RuntimeError(f"IK infeasible at {far_target_pos}")
```
```

---

### ## Waypoint Registry (grows per chunk)

Validated multi-step transit patterns for scenes where direct motion fails.

| Scenario | Pattern | First Seen In | Notes |
|----------|---------|---------------|-------|
| ---      | ---     | ---           | ---   |

<!-- PLACEHOLDER: Registry starts empty. Populated by coordinator after each chunk. -->

<!-- Format: scenario description | waypoint pattern used | first task | why default 3-waypoint failed -->

---

### ## Anti-Patterns

- Single `goto\_pose` across the whole scene - clips clutter. Use waypoints.
- Lift waypoint at same z as grasp\_pos - drags the object sideways.
- Descending straight into a container without a rim-clearance check - grasp collides with walls.
- Forgetting `for \_ in range(3): get\_observation()` after `open\_gripper()` - object doesn't settle; next observation is stale.
- Using `goto\_pose` in far workspace where `solve\_ik` + `move\_to\_joints` is required.